

Target User Attack on Federated Recommender Systems with Proxy Items and Its Countermeasure

DONGYI ZHENG, Sun Yat-sen University, Pengcheng Laboratory, China

LINGZHI WANG, Harbin Institute of Technology (Shenzhen), China

JIYUAN FENG, Pengcheng Laboratory, Harbin Institute of Technology (Shenzhen), China

YONGHONG TIAN, Peking University, Pengcheng Laboratory, China

NONG XIAO, Sun Yat-sen University, China

QING LIAO*, Harbin Institute of Technology (Shenzhen), Pengcheng Laboratory, China

The decentralized nature of federated recommender systems (FedRecs) introduces new security risks, particularly target item promotion attacks, where attackers may attempt to manipulate the ranking and exposure rate of target items in users' recommendation results. However, existing attacks typically promote each target item broadly to all users, which will compromise attack stealthiness while yielding only marginal gains in effectiveness. To address these issues, we propose a novel **proxy-item** based **target user** attack method on FedRecs, called **ProitTUA**, which selectively promotes each target item to a relevant user group rather than all users. The key novelty is to associate each target item with a relevant user group via category-wise proxy items. ProitTUA consists of two key stages: (1) item latent category inference, which infers latent item categories by leveraging auxiliary domain modeling and attack domain adaptation; and (2) proxy-item-based attack construction, which identifies proxy items for each category and then aligns each target item's embedding with its corresponding proxies. Extensive experiments show ProitTUA achieves superior attack effectiveness and stealthiness. To mitigate the potential threat posed by ProitTUA, we further design a defense mechanism based on gradient dynamics analysis. Experimental results demonstrate its effectiveness in reducing ProitTUA's impact on FedRecs.

CCS Concepts: • **Information systems** → **Recommender systems**.

Additional Key Words and Phrases: Federated Recommender System, Poisoning Attack and Defense, Target User Attack

ACM Reference Format:

Dongyi Zheng, Lingzhi Wang, Jiyuan Feng, Yonghong Tian, Nong Xiao, and Qing Liao. 2025. Target User Attack on Federated Recommender Systems with Proxy Items and Its Countermeasure. *J. ACM* 37, 4, Article 111 (December 2025), 36 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*Corresponding author.

Authors' Contact Information: Dongyi Zheng, zhengdy23@mail2.sysu.edu.cn, Sun Yat-sen University, Pengcheng Laboratory, Shenzhen, China; Lingzhi Wang, Harbin Institute of Technology (Shenzhen), Shenzhen, China, wanglingzhi@hit.edu.cn; Jiyuan Feng, Pengcheng Laboratory, Harbin Institute of Technology (Shenzhen), Shenzhen, China, fengjy@stu.hit.edu.cn; Yonghong Tian, Peking University, Pengcheng Laboratory, Shenzhen, China, yhtian@pku.edu.cn; Nong Xiao, Sun Yat-sen University, Shenzhen, China, nongxiao@nudt.edu.cn; Qing Liao, Harbin Institute of Technology (Shenzhen), Pengcheng Laboratory, Shenzhen, China, liaqing@hit.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2025/12-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Recommender systems [20, 53] are widely deployed in many online scenarios (e.g., e-commerce and social media) to help users discover what they are interested in from mass information. Traditional recommender systems [10, 24] require collecting user's personal data to train recommendation models in a centralized way. With the growing concerns about privacy and the new privacy protection regulations (e.g., GDPR [47] in the European Union and CCPA [1] in the United States), federated recommender systems (FedRecs) [7, 27, 33, 58] have recently emerged as a privacy-preserving solution to collaboratively learn a recommendation model among personal devices without uploading user's raw data.

However, the decentralized and open nature of federated recommender systems makes them inherently vulnerable to model poisoning attacks. Due to not all clients can be trusted, malicious clients or users¹ can participate in training and inject manipulated gradients to corrupt the global model. Among various threats, target item promotion attacks [38, 63] are particularly concerning, as they attempt to manipulate recommendation outputs by promoting specific items, potentially leading to unfair competition.

Recently, many studies [19, 57, 62] have explored various forms of target item promotion attacks to reveal the vulnerabilities of federated recommender systems. Existing attacks can be categorized into user embedding approximation-based attacks and item popularity enhancement-based attacks [62]. The user embedding approximation-based attacks [19, 37, 38, 41, 57] generate malicious gradients by maximizing the predicted scores of the target item. It first simulate universal user embeddings to approximate user preferences and then optimize the target item embedding to achieve high predicted scores on these simulated user embeddings. For example, A-hum [37] samples user embeddings from Gaussian distribution. PSMU [57] constructs synthetic user-item interaction histories and then learns user embeddings from them. The item popularity manipulation-based attacks generate malicious gradients by boosting the popularity of the target item [55, 62, 63]. Early efforts such as PipAttack [63] train a popularity classifier and optimize the target item's embedding to maximize its predicted popularity score. Recently, some methods first mine potential popular items and then align the target item's embedding with them. For example, PoisonFRS [55] computes the similarity between each item embedding and the global item centroid, then identifies items with high similarity as popular. PIECKIPE [62] computes embedding updates between consecutive communication rounds and flags items with significant changes as popular.

While existing target item promotion attacks can effectively boost target item exposure, they typically aim to promote target items to the entire user base to gain the most exposure, which is referred to as an all-user attack task, as illustrated in Figure 1. However, we argue that attacking all users is penny-wise and pound-foolish: as it may yield only marginal gains in effectiveness while compromising the stealthiness of attack. On one hand, a large portion of the exposed users may have no intrinsic interest in the target item. Since actual interactions (e.g., clicks or purchases) are unlikely even if the target item is recommended, attacking these users may yield only marginal effectiveness gains. On the other hand, large-scale exposure introduces significant perturbations to the recommendation results of many users, which can increase the risk of detection by defense mechanisms, thereby compromising the attack's stealthiness. Therefore, we propose a new target-user attack task, which aims to focus attack efforts on users likely to be interested in the target items, rather than targeting the entire user base, thereby maintaining attack effectiveness while enhancing stealthiness.

In federated recommender systems, realizing the target-user attack objective is inherently challenging. Existing all-user attack methods typically maximize the predicted score of each target item

¹In this article, client and user are treated as equivalent, assuming each client represents a single user for privacy reasons.

across the entire user population, which implicitly promotes exposure in a user-agnostic manner. While such strategies can inflate overall exposure, they fail to distinguish between users who are genuinely interested in the target item and those who are not, thereby lacking the capability for selective promotion. More fundamentally, the federated setting introduces two intrinsic technical challenges that prevent direct realization of target-user attacks: (1) the attacker cannot explicitly identify the potential target user set and non-target users due to strict privacy constraints, and (2) the attacker cannot compute or optimize any user-item relevance score because user embeddings are private and inaccessible. These limitations remove the critical supervision signals required for user-level targeting, rendering naive or direct optimization toward target users infeasible in federated environments.

To address the above issues, we propose a **proxy-item** based **target user attack** method for federated recommender systems, termed **ProitTUA**. ProitTUA can selectively promote each target item to its potential target users rather than all users, as shown in Figure 1. The key idea of ProitTUA is inspired by an interesting observation: popular item is more likely to be recommended due to its inherent popularity, and this exposure is further biased toward specific users whose preferences align with the item's category.², as shown in Table 2. Based on this finding, we propose an assumption that if a target item is embedded close to popular items of the same category in latent space, it is more likely to be recommended to users who like that category than to those who don't. With this assumption, ProitTUA constructs category-wise proxy items as bridges that link target items to their potential user groups. ProitTUA consists of two main stages: (1) Item latent category inference, which leverages an auxiliary public dataset to infer latent item categories under limited category prior knowledge; and (2) Proxy-item based attack construction, which leverages the inferred item categories to generate category-wise proxy items and aligns target items' embeddings with their corresponding proxy items to craft poisoned gradients. By aligning each target item's embedding with category-consistent proxy items, ProitTUA selectively increase its exposure to users who have intrinsic interest in the corresponding category, thereby maintaining attack effectiveness while enhancing stealthiness.

To evaluate the effectiveness and stealthiness of ProitTUA, we conduct extensive experiments on two widely used federated recommendation frameworks, FedNCF and FedDMF, across three real-world datasets: MovieLens-1M, Amazon Book, and Steam. The results reveal the inherent vulnerability of FedRecs to ProitTUA and demonstrate that ProitTUA consistently outperforms existing attack methods. ProitTUA achieves significantly higher attack precision, lower attack error rate, and reduced attack cover rate, confirming its superior effectiveness. Moreover, comprehensive evaluations show that ProitTUA maintains high recommendation quality and sustains effective attack performance under various defense mechanisms, highlighting its high stealthiness.

To further mitigate the threat posed by ProitTUA, we propose a defense mechanism based on gradient dynamics analysis. Motivated by normal items' stable gradient evolution patterns during training, our defense first identifies items with abnormal gradient fluctuations, then detects suspicious clients that predominantly contribute to these anomalous items and finally excludes them from the aggregation process. Experimental results demonstrate that this defense mechanism effectively mitigates ProitTUA's manipulation of target items.

To summarize, the main contributions of this article are as follows:

- We propose a proxy-item based target user attack method on FedRecs, termed **ProitTUA**³, which leverages category-wise proxy items to establish reasonable attack connections

²A category denotes a high-level semantic label, e.g., war, sci-Fi for movies, or history, technology for books.

³Our code and dataset will be released upon acceptance of this article.

Federated Training

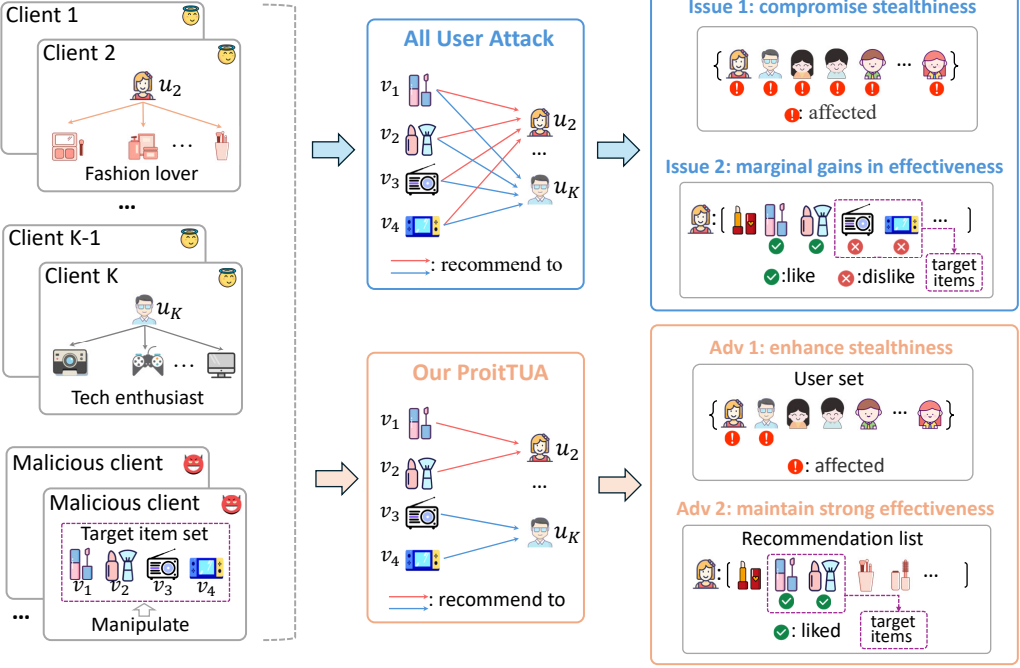


Fig. 1. Illustration of previous all user attack and our ProfitTUA on FedRecs.

between target items and their relevant user groups. ProfitTUA enables selective promotion of each target item to its semantically relevant user group, rather than all users.

- ProfitTUA consists of two main stages, i.e., (1) item latent category inference and (2) proxy-item based attack construction. The former stage infers latent semantic categories of items under limited category prior knowledge via auxiliary domain modeling and attack domain adaptation. The latter stage crafts poisoned gradients for each target item through category-wise proxy item mining and proxy-guided poisoned gradient generation.
- Extensive experiments demonstrate that ProfitTUA not only maintains high attack effectiveness, but also enhances stealthiness by effectively evading existing defenses and preserving overall recommendation quality.
- To mitigate the threat of ProfitTUA, we propose a defense mechanism based on gradient dynamics analysis. Experimental results demonstrate that the proposed defense effectively reduces ProfitTUA's impact on federated recommendation systems.

The remainder of this article is organized as follows. Section 2 provides a comprehensive review of existing attack strategies and defense mechanisms in FedRecs. Section 3 introduces the foundational concepts of FedRecs and defines the attack problem. Section 4 details the technical design of our ProfitTUA. Section 5 presents extensive experimental results and analysis, evaluating both the effectiveness and stealthiness of ProfitTUA. In Section 6, we discuss the defense method tailored to counter ProfitTUA and evaluate its performance. Finally, Section 7 summarizes our contributions and discusses promising directions for future research.

2 Related Work

In this section, we briefly review the literature on three related topics: federated recommendation systems, attacks against federated recommendation systems, defenses for federated recommendation. Other involved topics such as the development of general recommendation systems and federated learning can be referred to corresponding surveys [3, 9, 11, 65].

2.1 Federated Recommendation System

FedRec systems enable the training of global recommendation models while preserving user privacy by keeping sensitive data on local devices. The pioneering work, Federated Collaborative Filtering [2], was the first to adopt federated learning for recommendation, setting the foundation for subsequent research. Following this, a range of FedRec models [26, 30] introduced adaptations of distributed matrix factorization, aiming to balance privacy preservation with effective recommendation. Despite their contributions, FedRec methods continue to face challenges such as cold start [48] and data sparsity [64].

With the advancement of deep learning, more sophisticated FedRec frameworks have been developed, incorporating techniques such as meta-learning [28], graph neural networks [31, 50], neural collaborative filtering [36], reinforcement learning [43], and contrastive learning [52]. These techniques have substantially enhanced recommendation accuracy, demonstrating their potential to mitigate challenges such as cold start and data sparsity. However, the secure and robust deployment of FedRec remains a challenge due to inherent vulnerabilities, particularly in the face of adversarial attacks [14].

2.2 Attacks against Federated Recommendation System

The decentralized and distributed nature of FedRec systems makes them particularly susceptible to various forms of attacks, especially poisoning attacks, where adversaries attempt to manipulate the recommendation model by distorting either data or model updates [41, 57]. Poisoning attacks can be broadly categorized into two types: data poisoning [4] and model poisoning [15, 34].

2.2.1 Data Poisoning Attacks. Data poisoning attacks manipulate the training dataset by injecting malicious interactions into user data [16, 21, 35]. When a recommender model is trained on poisoned data, it can be misled to produce abnormal recommendation results. Most data poisoning attacks [16, 21, 35, 49, 61] have been designed for centralized recommendation systems. However, in federated recommender systems, where users' raw data are never shared and attackers lack access to other users' data [23, 33], the effectiveness and applicability of such attacks are significantly constrained. As a result, only a few studies [46] have explored data poisoning in the context of FedRecs, and their effectiveness remains rather constrained.

2.2.2 Model Poisoning Attacks. Due to the decentralized nature of FedRecs, they are more vulnerable to model poisoning attacks [62]. These attacks disrupt the recommendation model by allowing malicious clients to upload poisoned gradients to the central server. Based on the attackers' goals, model poisoning attacks can be broadly classified into two categories: *system performance destruction attacks* [51], and *target item promotion attacks* [63]. The *system performance destruction attacks* aim to reduce recommendation accuracy and thereby undermine the utility of the system. For example, FedAttack [51] increases the prediction scores for items that users are unlikely to prefer by hard negative sampling. ClusterAttack[56] perturbs the normal distribution of item embeddings through clustering techniques. In contrast, the *target item promotion attacks* have attracted more research interest and is also the focus of this work. By altering the rank of a target item, malicious users can increase its exposure and potentially gain more economic

benefits [55]. Existing *target item promotion attacks* can be roughly grouped into two approaches [62]: (1) user embedding approximation-based attacks [19, 37, 38, 41, 57] and (2) item popularity enhancement-based attacks [55, 62, 63].

User Embedding Approximation-Based Attacks. This line of attack generates malicious gradients by maximizing the predicted scores of the target item, first simulating user embeddings and then optimizing the target item's embedding to achieve the maximum predicted scores from these simulated users. Early efforts such as A-ra [37] approximate user embeddings by sampling from a Gaussian distribution and then directly maximize the target item's predicted scores on these synthetic embeddings. A-hum [37] improves upon A-ra by incorporating hard user mining to refine the sampled embeddings, thereby better approximating the real user distribution. FedRecAttack [38] simulates user behavior more explicitly by leveraging real interaction data (e.g., item click logs) to create more faithful approximations. PSMU [57] constructs synthetic user-item interaction histories on malicious clients and then learns simulated user embeddings from them, which are subsequently used to optimize the target item's score. HMTA [41] thinks that attacking users who dislike the target item can be more effective, and further shifts the perspective by taking target items as negative samples to train simulated user embeddings. PIECKUEA [62] proposes to approximate user embeddings by modeling the embedding distribution of popular items, under the assumption that users interacting with such items are more representative. More recently, StairClimbing [19] first trains user embeddings based on malicious client data and then constructs a Gaussian distribution from these trained user embeddings to sample simulated user embeddings. Since these methods aim to maximize exposure among all users, their simulated user embeddings capture diverse preferences. Therefore, maximizing one target item's prediction score across these user embeddings will promote it widely among all users.

Item Popularity Enhancement-Based Attacks. This line of attack produces malicious gradients by boosting the popularity of the target item. Early efforts such as PipAttack [63] first trains a popularity classifier using global item popularity statistics, and then optimizes the target item's embedding to maximize its predicted popularity score under the trained popularity classifier. Recently, some methods first mine potential popular items and then aligns the target item's embedding with them. For example, PoisonFRS [55] thinks popular items tend to aggregate near the center of the embedding space and identifies popular items as those whose embeddings are most similar to the global mean item embedding. PIECKIPE [62] analyzes item embedding update across consecutive communication rounds and identifies popular items as those whose embeddings exhibit significant changes. Since these methods also aim to maximize exposure among all users, the popularity information they utilize is globally prevalent, which will lead to the target items being widely promoted among all users.

2.3 Defenses for Federated Recommendation Systems

Defense mechanisms in general federated learning have been extensively explored in the machine learning context [22, 29, 32]. These methods typically assume that malicious updates are both statistically deviant and limited in number. Accordingly, defenses include gradient normalization [44], coordinate-wise median or trimmed mean aggregation [54], and similarity-based outlier filtering [6, 17]. While effective in general federated learning, these techniques struggle in FedRecs, where the set of items involved in each client's gradient updates varies significantly, making it difficult to apply standard statistical filtering methods.

To address this challenge, recent works have proposed defense strategies tailored for FedRecs. These methods can be broadly categorized into server-side and client-side defenses. Server-side defenses operate during global model aggregation to suppress suspicious updates. For example, CrossEU [13] inspects the item sets involved in each client's uploaded gradients. It flags clients

as suspicious if the set of updated items fluctuates significantly across communication rounds, or multiple clients consistently upload gradients involving highly similar item sets, suggesting coordinated behavior. HiCS [57] detects and mitigates malicious item gradients by applying hierarchical clipping and sparsified aggregation. Client-side defenses act during local training to increase the attack difficulty of malicious clients. For example, Regular [62] introduces two regularization terms during benign clients' local training to restrict the popularity enhancement of target items and limit user embedding approximation, thereby weakening the ability of malicious clients to generate effective poisoned gradients.

In parallel, a few defenses have been developed targeting specific attack types in federated recommendation. For example, GDMPD[60] detects malicious clients in visually-aware FedRecs by identifying abnormal image gradients that result from manipulated visual features. rCLFedRec[59] enhances robustness in contrastive learning-based FedRecs by introducing regularization terms to maintain the distance among item embeddings with different popularity levels. UNION [56] disrupts cluster-aware attacks by obfuscating semantic cluster structures. Although these defenses show promise in their respective scenarios, they are inherently task-specific and rely on unique model structures or auxiliary modalities. As such, they cannot be readily generalized to standard federated recommendation settings, where such assumptions may not hold.

3 Preliminary

In this section, we provide the framework of the federated recommendation systems and the fundamental settings of the attack objective. The federated recommendation framework used in this article is the same as previous FedRecs attack works [37, 41, 57, 62]. The meaning of the most important notations are listed in Table 1.

Table 1. List of important notations.

Symbol	Description
A	Auxiliary domain with semantic-labeled items
T	Attack domain (i.e., federated recommendation domain)
$\mathcal{D}_A$	User-item interaction data in auxiliary domain
$\mathcal{V}_A$	Item set in the auxiliary domain
$\mathcal{V}_T$	Item set in the attack domain
$\mathcal{V}^{\text{tar}}$	Target item set
$\mathcal{V}_u^{\text{rec}}$	The recommendation list of user u
c	The category identifier
$\mathcal{V}_T^{\text{seen}}$	Items with category labels visible to the malicious client
$\mathcal{V}_T^{\text{unseen}}$	Items with category labels invisible to the malicious client
$\mathcal{U}$	Set of benign clients
$\tilde{\mathcal{U}}$	Set of malicious clients
$\mathbf{U}$	User embedding matrix
$\mathbf{V}$	Item embedding matrix
Θ	Prediction layer parameters of the recommendation model
$H_\Phi(\cdot)$	Classifier parameterized by Φ
$G_\Psi(\cdot, \cdot)$	Matcher parameterized by Ψ

3.1 Federated Recommendation System

The federated recommendation system (FedRec) is a distributed recommendation paradigm that enables multiple clients to collaboratively train a recommendation model without sharing their private data. In the federated learning setting, a central server coordinates multiple clients. The parameters of FedRec are divided into public (stored on the server and shared with all clients) and private (stored locally on the client and inaccessible externally). User embeddings, denoted by $\mathbf{U}$, are private parameters. In contrast, item embeddings ($\mathbf{V}$) and the prediction layer parameters Θ are public. Below, we define and explain the key notations used in FedRec.

Notations in FedRec. Let $\mathcal{U}$ and $\mathcal{V}$ represent the sets of users and items, respectively. Each client u_i owns a private local dataset $\mathcal{D}_i$, consisting of implicit feedback tuples (u_i, v_j, r_{ij}) . Here, r_{ij} equals 1 if u_i has interacted with v_j , and 0 otherwise. Define $\mathcal{V}_i^+$ as the set of items that u_i has interacted with, and $\mathcal{V}_i^-$ as the set of non-interacted items. The local recommender's objective is to predict interaction probabilities $\hat{r}_{ij}$ between u_i and item v_j , and ultimately recommend the top- K items with the highest predicted scores.

Workflow of a Standard FedRec System. The training process begins with the central server initializing all public parameters, while each client initializes its corresponding private parameters. Then, the central server randomly selects a subset of clients at communication epoch t , denoted as $\mathcal{U}_t$, to participate in the training and provides them with the public parameters Θ^t and $\mathbf{V}^t$. Each selected client combines these public parameters with their local private parameters, $\mathbf{u}_i^t$, to form a personalized local recommender model. This local recommender is then updated by optimizing the following loss function on the client's local dataset, $\mathcal{D}_i$:

$$\mathcal{L}^{rec} = - \sum_{(u_i, v_j, r_{ij}) \in \mathcal{D}_i} [r_{ij} \log \hat{r}_{ij} + (1 - r_{ij}) \log(1 - \hat{r}_{ij})] \quad (1)$$

After a few local epochs, the private parameters of client u_i are:

$$\mathbf{u}_i^{t+1} = \mathbf{u}_i^t - lr \nabla \mathbf{u}_i^t \quad (2)$$

where lr is the learning rate.

The gradients of the public parameters, $\nabla \Theta_i^t$ and $\nabla \mathbf{V}_i^t$, are then uploaded to the central server. The server aggregates the gradients from all selected clients and updates the public parameters as follows:

$$\mathbf{V}^{t+1} = \mathbf{V}^t - lr \sum_{u_i \in \mathcal{U}_t} \nabla \mathbf{V}_i^t, \quad \Theta^{t+1} = \Theta^t - lr \sum_{u_i \in \mathcal{U}_t} \nabla \Theta_i^t \quad (3)$$

3.2 Problem Formulation and Attack Objective

We consider a realistic model poisoning threat in federated recommender systems (FedRecs), where an attacker aims to *selectively* promote a set of target items $\mathcal{V}^{\text{tar}}$ to users with potential interest. We term this fine-grained promotion strategy the *target user attack*, which differs from existing approaches that expose target items to all users. By limiting exposure to relevant users, the target user attack enhances *stealthiness* while preserving attack *effectiveness*.

Attack Prior Knowledge. Following prior work [19, 57, 62], we assume the attacker controls a small set of malicious clients $\tilde{\mathcal{U}}$ and has access to:

- the learning rate lr used in model updates,
- the shared model architecture in FedRecs,
- the global item embedding matrix $\mathbf{V}^t$ and shared parameters Θ^t at round t ,
- category metadata for a subset of items (including target items), e.g., genre or brand.

The attacker can only upload poisoned gradients via $\tilde{\mathcal{U}}$ during communication rounds. Specifically, the attacker cannot access any private data from benign clients, including interaction histories, item category metadata, personalized user embeddings, or local gradients.

Revisiting Existing Objective. Most prior poisoning attacks follow an all user attack objective, aiming to maximize the predicted score $f(u, v)$ of each target item v across all users $u \in \mathcal{U}$, thereby promoting global exposure:

$$\min_{\{\nabla_v\}_{v \in \mathcal{V}^{\text{tar}}}} \mathcal{L}_{\text{all-user}} = - \sum_{v \in \mathcal{V}^{\text{tar}}} \sum_{u \in \mathcal{U}} f(u, v) \quad (4)$$

While this objective may inflate exposure, it fails to distinguish between users who are actually interested in v and those who are not. This will lead to large-scale user attacks, potentially compromising the attack's stealthiness while yielding only limited gains in effectiveness.

Our Proposed Objective. We instead propose a fine-grained target user attack objective. For each target item $v \in \mathcal{V}^{\text{tar}}$, we define:

- $\mathcal{U}_v^+$: the *target users* for item v , i.e., users likely to prefer v ;
- $\mathcal{U}_v^- = \mathcal{U} \setminus \mathcal{U}_v^+$: the *non-target users* for item v , i.e., users unlikely to prefer v .

We formalize our attack objective as:

$$\min_{\{\nabla_v\}_{v \in \mathcal{V}^{\text{tar}}}} \mathcal{L}_{\text{tar-user}} = - \sum_{v \in \mathcal{V}^{\text{tar}}} \left[\sum_{u \in \mathcal{U}_v^+} f(u, v) - \lambda \sum_{u \in \mathcal{U}_v^-} f(u, v) \right] \quad (5)$$

where $\lambda > 0$ balances the trade-off between promoting target item v to its target users while suppressing its exposure to non-target users. By achieving this attack objective, each target item can be promoted specifically to its relevant user group rather than to unrelated users, thereby ensuring high attack effectiveness. In addition, by reducing the impact on non-target users, the approach helps preserve overall recommendation quality and lowers the risk of detection by defense mechanisms, thus enhancing the stealthiness of the attack.

In the following section, we introduce a surrogate optimization method based on category-specific proxy items, which enables fine-grained promotion under limited attacker knowledge.

4 Methodology

In this section, we introduce our proposed attack method **ProitTUA** (Proxy-item based Target User Attack), which implements a fine-grained item promotion strategy on federated recommender systems.

4.1 Motivation and Overview

As analyzed in Section 3.2, existing poisoning methods aim to attack all users to achieving the maximum exposure, which leads to compromised stealthiness and may yield only limited gains in effectiveness. To overcome this, we define a more desirable objective: promote each target item v to its target users, while suppress its exposure to non-target users, as formulated in Equation (5).

However, optimizing this objective in practice faces two major obstacles: (1) the attacker cannot identify target users $\mathcal{U}_v^+$ and non-target users $\mathcal{U}_v^-$ due to privacy constraints, and (2) the attacker cannot compute any user-item score $f(u, v)$, since user embeddings are private and inaccessible. These limitations render direct optimization infeasible.

Motivation. Despite the above restrictions, we are inspired by an interesting observation that *popular item is more likely to be recommended due to its inherent popularity, and this exposure is further biased toward specific users whose preferences align with the item's category*, as shown in Table 2. This category-specific exposure bias provides an opportunity to guide promotion indirectly,

through the alignment of target items with category-specific popular items. Based on this finding, we propose an assumption as the foundation of our method: *If a target item is embedded close to popular items of the same category in latent space, it is more likely to be recommended to users who like that category than to those who don't.*

To further clarify the causal mechanism behind proxy-item alignment, we analyze the inherent connections from the perspectives of user preference structure and item feature geometry. First, users usually exhibit category-level preferences over items: they typically favor items belonging to their preferred categories, which is a fundamental prior in recommender systems [8, 12]. Second, popular items generally have higher recommendation scores and are naturally more likely to be recommended to users [40]. Third, aligning a target item to its category-consistent popular proxy items enables the target item to inherit the category-wise preference from proxies. This process establishes a clear causal path: item category semantics $\rightarrow$ item embedding distribution $\rightarrow$ user preference matching, which allows our method to achieve targeted exposure without accessing any private user embeddings.

Table 2. Exposure bias across different types of popular items: popular items in a user's favorite category (FavCat) achieve higher hit rates than those in non-favorite categories (NonFavCat). Note: A user's favorite category is defined as one with over 10 interactions.

Dataset	Popular Types	FedNCF			FedDMF		
		Hit@1	Hit@5	Hit@10	Hit@1	Hit@5	Hit@10
Steam	FavCat	0.19	0.56	0.75	0.22	0.61	0.80
	NonFavCat	0.11	0.31	0.50	0.13	0.36	0.55
MovieLens	FavCat	0.23	0.60	0.78	0.25	0.65	0.82
	NonFavCat	0.12	0.34	0.52	0.14	0.38	0.57

Overview. With this assumption, we propose ProitTUA, a proxy-item based target user attack method that focuses the attack on relevant target users while reducing unintended exposure to non-target users. Specifically, ProitTUA identifies category-wise proxy items and embeds each target item close to its corresponding proxies, thereby selectively promoting each target item to a targeted subset of users rather than the entire user base. As illustrated in Figure 2, ProitTUA comprises two main stages:

- **Item Latent Category Inference (ILCI):** Due to data isolation in FedRecs, the malicious client can only access category labels for a small subset of local items, making it infeasible to directly mine category-wise popular items as proxies. To address this limited category prior knowledge, we introduce an *auxiliary domain* that contains a public dataset with category-labeled items. We train a classification model in this auxiliary domain and transfer it to the *attack domain*—the target scenario where the attack is deployed—to enable latent category inference for those items whose category information are unavailable.
- **Proxy-item based Attack Construction (PIAC):** Based on the inferred categories, we first identify in-class popular items as proxy items and then align each target item's embedding with those of its category proxies to generate poisoned gradients. This construction indirectly boosts exposure among users favoring the associated category, thereby realizing the dual goals of effectiveness and stealthiness.

In the following subsections, we elaborate on the design of each stage, detailing how ProitTUA coordinates them to achieve the target user attack objective.

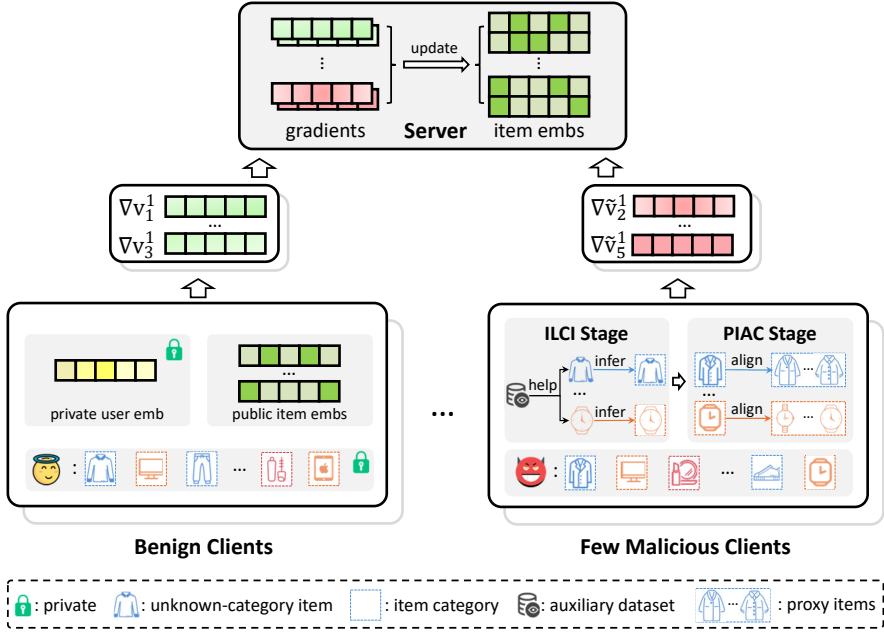


Fig. 2. Overview of **ProitTUA**. The malicious client proceeds in two stages: (1) *Item Latent Category Inference (ILCI)*, which leverages a auxiliary dataset to assist in inferring the potential categories of items under strict data isolation; and (2) *Proxy-item Based Attack Construction (PIAC)*, which utilizes category-specific proxy items to generate poisoned gradients that selectively promote each target item toward relevant user groups. This two-stage design enables fine-grained target user attacks that are both effective and stealthy.

4.2 Item Latent Category Inference

Due to the inherent data isolation in federated recommender systems, malicious clients have access only to category labels of items locally available, whereas category labels of items from other benign clients are inaccessible. However, category information is essential for both effective and stealthy item promotion, as it allows attackers to mine category-wise proxy items that associate target items with semantically relevant user groups. To address this limited category prior knowledge, we leverage a public dataset with available category labels to enable category inference for all items under privacy constraints, laying the foundation for the subsequent proxy-item based attack construction. For clarity, we refer to the public dataset as the *auxiliary domain* (A), and the target scenario where the attack is deployed as the *attack domain* (T). The inference stage comprises two steps: auxiliary domain modeling and attack domain adaptation. Notably, the auxiliary domain must be from the same type of recommendation task as the attack domain. The reasons are threefold: First, the item categories of the two domains are naturally consistent, and their high-level semantics can be well aligned. Second, user preference patterns are highly correlated under the same recommendation task, as they essentially belong to the same recommendation scenario. Third, when using the same recommendation model architecture, item feature distributions are well correlated, thus supporting reliable knowledge transfer across domains. If the two domains differ significantly (e.g., using a book dataset to assist movie recommendation), the domain gap will increase sharply, making knowledge migration completely ineffective. Therefore, we choose public datasets from the same domain and task as the auxiliary domain to ensure feasible and effective knowledge transfer.

4.2.1 Auxiliary Domain Modeling. This step aims to train a transferable category classifier that enables inferring the categories of items with invisible labels in the attack domain (denoted as $\mathcal{V}_T^{\text{unseen}}$). To ensure semantic transferability between the auxiliary and attack domains, we assume that both domains involve the same recommendation content, such as movies or books. Specifically, items in both domains are assumed to share a similar set of latent categories—for instance, movies across different datasets commonly include categories such as war, science fiction, and others. In this step, we first introduce the setup of the auxiliary domain and then present the design of the classifier. To enhance the classifier’s generalization ability in the attack domain, we further introduce domain space alignment, and a category matcher.

Auxiliary Domain Setup. We denote the auxiliary interaction data as $\mathcal{D}_A = \{(u_i, v_j, r_{ij})\}$, where $r_{ij} \in \{0, 1\}$ indicates whether user u_i has interacted with item v_j . The item set and user set are denoted by $\mathcal{V}_A$ and $\mathcal{U}_A$, respectively. Each item $v_j \in \mathcal{V}_A$ is associated with a category label c_j . The recommendation model in the auxiliary domain adopts the same architecture as that in the attack domain. It consists of a user embedding matrix $\mathbf{U}_A \in \mathbb{R}^{|\mathcal{U}_A| \times d}$, an item embedding matrix $\mathbf{V}_A \in \mathbb{R}^{|\mathcal{V}_A| \times d}$, and a prediction layer parameterized by Θ_A .

The recommendation model is optimized via the standard binary cross-entropy:

$$\mathcal{L}_A^{\text{rec}} = - \sum_{(u_i, v_j, r_{ij}) \in \mathcal{D}_A} [r_{ij} \log f_{\Theta_A}(\mathbf{u}_i, \mathbf{v}_j) + (1 - r_{ij}) \log(1 - f_{\Theta_A}(\mathbf{u}_i, \mathbf{v}_j))]. \quad (6)$$

Category Classifier. The classifier provides the fundamental ability to predict the category of an item. It is defined as a category probability prediction function $H_\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^C$, which maps an item embedding to a C -dimensional category probability vector.

Let $\mathbf{v}_i \in \mathbb{R}^d$ denote the embedding of item v_i . The classifier is designed as a L -layer Multi-Layer Perceptron (MLP) [45], denoted $H_\Phi(\cdot)$ with parameters Φ . It predicts a class distribution $\hat{\mathbf{y}}_i \in \mathbb{R}^C$ using:

$$\hat{\mathbf{y}}_i = H_\Phi(\mathbf{v}_i) = \text{softmax}\left(\mathbf{W}^{(L+1)} \cdot \phi(\dots \phi(\mathbf{W}^{(1)} \mathbf{v}_i + \mathbf{b}^{(1)}) \dots) + \mathbf{b}^{(L+1)}\right), \quad (7)$$

where $\phi(\cdot)$ is the non-linear activation function, typically ReLU or GELU.

The classifier is trained using the standard cross-entropy loss over labeled samples in the auxiliary domain:

$$\mathcal{L}_A^{\text{cla}} = - \sum_{v_i \in \mathcal{V}_A} \sum_{c=1}^C y_{ic} \log \hat{y}_{ic}, \quad (8)$$

where $y_{ic} \in \{0, 1\}$ denotes the one-hot ground-truth label of item v_i for category c .

Domain Space Alignment. To further enhance the classifier’s transferability, we mitigate distributional discrepancies between the two domains via domain space alignment strategies, including both global space alignment and category subspace alignment.

(1) **Global Space Alignment:** We align the marginal embedding distributions of domains A and T using the Maximum Mean Discrepancy (MMD) [39], which measures the distance between empirical kernel means of two distributions in a Reproducing Kernel Hilbert space (RKHS) [5]:

$$\mathcal{L}_{\text{align}}^{\text{global}} = \text{MMD}^2(\mathbf{V}_A, \mathbf{V}_T) = \left\| \frac{1}{|\mathcal{V}_A|} \sum_{i=1}^{|\mathcal{V}_A|} \phi(\mathbf{v}_i^A) - \frac{1}{|\mathcal{V}_T|} \sum_{j=1}^{|\mathcal{V}_T|} \phi(\mathbf{v}_j^T) \right\|_2^2, \quad (9)$$

where $\phi(\cdot)$ is a feature mapping that projects item embeddings into a high-dimensional RKHS induced by a kernel function.

(2) **Category Subspace Alignment:** In the auxiliary domain A , let $\mathcal{V}_A^c$ be the set of items in category c . In the attack domain T , $\mathcal{V}_T^{\text{seen}}$ denotes the malicious client’s local items with known

categories, and $\mathcal{V}_T^{seen,c}$ is the subset of these items belonging to category c . We define a category-level contrastive loss based on category centers. For each class c , let μ_A^c and μ_c^T denote the mean embedding (i.e., category center) of class- c items from the $\mathcal{V}_c^A$ and $\mathcal{V}_T^{seen,c}$, respectively. The category subspace alignment loss is defined as:

$$\mathcal{L}_{align}^{sub} = \sum_c \log \frac{\exp(-\|\mu_A^c - \mu_c^T\|^2/\tau)}{\sum_{c' \neq c} \exp(-\|\mu_A^c - \mu_{c'}^T\|^2/\tau)} \quad (10)$$

where τ is a temperature hyperparameter to scale the similarity scores.

The overall domain alignment loss is the weighted combination of global and subspace alignment:

$$\mathcal{L}_A^{align} = \lambda_{global} \cdot \mathcal{L}_{align}^{global} + \lambda_{sub} \cdot \mathcal{L}_{align}^{sub}. \quad (11)$$

Category Matcher. Due to the limited availability of category labels (i.e., $\mathcal{V}_T^{seen}$) in the attack domain, directly fine-tuning the classifier may suffer from label sparsity. To address this, we design a category matcher that predicts whether two items belong to the same category. The trained matcher can provide rich pairwise supervision signals to facilitate classifier fine-tuning and enhance its generalization in the attack domain.

Formally, the matcher is defined as a category similarity prediction function $G_\Psi : \mathbb{R}^d \times \mathbb{R}^d \rightarrow [0, 1]$, parameterized by Ψ . Given two item embeddings $\mathbf{v}_i$ and $\mathbf{v}_j$, the matcher predicts their category similarity:

$$\hat{g}_{ij} = G_\Psi(\mathbf{v}_i, \mathbf{v}_j) = \text{sigmoid}(\text{MLP}_\Psi([\mathbf{v}_i \parallel \mathbf{v}_j])), \quad (12)$$

where $[\mathbf{v}_i \parallel \mathbf{v}_j]$ denotes vector concatenation, MLP_Ψ is a multi-layer perceptron. The matcher is trained using binary supervision from the auxiliary domain:

$$\mathcal{L}_A^{mat} = - \sum_{(v_i \in \mathcal{V}_A, v_j \in \mathcal{V}_A)} [g_{ij} \log \hat{g}_{ij} + (1 - g_{ij}) \log(1 - \hat{g}_{ij})], \quad (13)$$

where $g_{ij} = 1$ if items v_i and v_j belong to the same class, and $g_{ij} = 0$ otherwise.

To prevent class imbalance and overfitting, training pairs are sampled uniformly from positive and negative classes, promoting generalizable relational semantics. This matcher will later serve as a pseudo label generator in the attack domain adaptation step.

Jointly Optimization. We jointly optimize the following losses in the auxiliary domain with task-specific weights:

$$\mathcal{L}_A = \mathcal{L}_A^{rec} + \lambda_{cla} \mathcal{L}_A^{cla} + \lambda_{mat} \mathcal{L}_A^{mat} + \lambda_{align} \mathcal{L}_A^{align} \quad (14)$$

where $\mathcal{L}_A^{rec}$, $\mathcal{L}_A^{cla}$, $\mathcal{L}_A^{mat}$, and $\mathcal{L}_A^{align}$ denote the losses of the recommendation model, classifier, matcher, and domain alignment, respectively. The hyperparameters λ_{cla} , λ_{mat} , and λ_{align} control the relative weights of these losses.

4.2.2 Attack Domain Adaptation. The goal of this step is to adapt the auxiliary-domain classifier to the unlabeled attack domain T , enabling accurate categorization of items in $\mathcal{V}_T^{unseen}$. Directly fine-tuning the classifier on the limited labeled set $\mathcal{V}_T^{seen}$ often leads to suboptimal performance due to label sparsity. To address this, we first adapt the matcher using both $\mathcal{V}_T^{seen}$ and auxiliary-domain items to construct category-aware training pairs. The adapted matcher then generates pairwise pseudo-labels to supervise the classifier, allowing it to generalize beyond $\mathcal{V}_T^{seen}$ and capture finer category structures under limited supervision.

Fine-tune Matcher. To adapt the matcher to the attack domain T , we fine-tune it using item pairs where at least one item comes from the labeled subset $\mathcal{V}_T^{seen}$ in the attack domain. Specifically,

each pair consists of (v_i, v_j) where $v_i \in \mathcal{V}_T^{\text{seen}}$ and $v_j \in \mathcal{V}_T^{\text{seen}} \cup \mathcal{V}_A$. Each pair is labeled as positive if the two items share the same category, and negative otherwise:

$$g_{ij} = \begin{cases} 1, & \text{if } c_i = c_j, \\ 0, & \text{otherwise,} \end{cases} \quad (15)$$

where c_i and c_j are the known categories of v_i and v_j , respectively.

The matcher G_Ψ then takes the concatenated embeddings $(\mathbf{v}_i, \mathbf{v}_j)$ as input and predicts a similarity score:

$$\hat{g}_{ij} = G_\Psi(\mathbf{v}_i, \mathbf{v}_j), \quad (16)$$

and is trained using binary cross-entropy:

$$\mathcal{L}_T^{\text{mat}} = - \sum_{(v_i \in \mathcal{V}_T^{\text{seen}}, v_j \in \mathcal{V}_T^{\text{seen}} \cup \mathcal{V}_A)} [g_{ij} \log \hat{g}_{ij} + (1 - g_{ij}) \log(1 - \hat{g}_{ij})]. \quad (17)$$

By jointly exploiting intra-domain and cross-domain label-aligned pairs, the matcher is equipped with the ability to compare item embeddings across heterogeneous domains. This capability is crucial for generating reliable pairwise pseudo-labels to supervise the classifier adaptation.

Fine-tune Classifier. After fine-tuning the matcher, we leverage it to construct pairwise soft pseudo labels that supervise the classifier's adaptation. Specifically, we consider item pairs (v_i, v_j) where $v_i \in \mathcal{V}_T^{\text{seen}}$ and $v_j \in \mathcal{V}_T^{\text{seen}} \cup \mathcal{V}_T^{\text{unseen}}$, covering both labeled-to-unlabeled and labeled-to-labeled combinations within the attack domain.

For each such pair, we query the matcher to obtain a soft similarity score:

$$s_{ij} = G_\Psi(\mathbf{v}_i, \mathbf{v}_j), \quad (18)$$

which reflects the likelihood that the two items belong to the same class. The classifier then predicts class probability distributions:

$$\mathbf{p} = H_\Phi(\mathbf{v}_i), \quad \mathbf{q} = H_\Phi(\mathbf{v}_j). \quad (19)$$

The adaptation objective encourages similar items (high s_{ij}) to yield similar predictions, and dissimilar items (low s_{ij}) to produce divergent outputs. Formally, the fine-tuning loss is defined as:

$$\begin{aligned} \mathcal{L}_T^{\text{cla}} = \sum_{(v_i, v_j) \in \mathcal{P}} & \left[s_{ij} \cdot (\text{KL}(\mathbf{p} \parallel \mathbf{q}) + \text{KL}(\mathbf{q} \parallel \mathbf{p})) \right. \\ & \left. + (1 - s_{ij}) \cdot (L_h(\text{KL}(\mathbf{p} \parallel \mathbf{q}), \delta) + L_h(\text{KL}(\mathbf{q} \parallel \mathbf{p}), \delta)) \right] \end{aligned} \quad (20)$$

where $\mathcal{P} = \{(v_i, v_j) \mid v_i \in \mathcal{V}_T^{\text{seen}}, v_j \in \mathcal{V}_T^{\text{seen}} \cup \mathcal{V}_T^{\text{unseen}}\}$ denotes the set of training pairs, and $L_h(x, \delta) = \max(0, \delta - x)$ is a hinge loss with margin δ that enforces confident separation for dissimilar pairs.

This pairwise fine-tuning procedure allows the classifier to generalize its decision boundaries to the unlabeled subset $\mathcal{V}_T^{\text{unseen}}$, under the guidance of matcher-derived semantic signals.

Category Inference. After adaptation, we apply the classifier H_Φ to infer the category label for each unseen item $v_j \in \mathcal{V}_T^{\text{unseen}}$. Specifically, we compute:

$$\hat{\mathbf{p}}_j = H_\Phi(\mathbf{v}_j), \quad \hat{c}_j = \arg \max_c \hat{p}_{jc}. \quad (21)$$

Each unseen item is thus assigned to a predicted category. We then merge these predictions with the seen items $\mathcal{V}_T^{\text{seen}}$, whose categories are already known, to construct category-specific item subsets:

$$\mathcal{V}_T^c = \{v_j \in \mathcal{V}_T^{\text{seen}} \mid c_j = c\} \cup \{v_j \in \mathcal{V}_T^{\text{unseen}} \mid \hat{c}_j = c\}, \quad \forall c \in \{1, \dots, C\}. \quad (22)$$

These category-specific item subsets form the foundation for subsequent proxy-item based attack construction stage.

4.3 Proxy-item based Attack Construction

After inferring the category label for each item in the attack domain, we now proceed to generate poisoned gradients for target item promotion. The key idea is to use category-wise proxy items—i.e., multiple representative popular items in each category—to bridge the connection between each target item and its corresponding user group. This stage consists of two steps: category-wise proxy item mining and proxy-guided poisoned gradient generation.

4.3.1 Category-wise Proxy Item Mining. To maintain effectiveness and enhance stealthiness, we design the attack to selectively promote each target item to users who are likely to be interested in it. Based on our previous finding (see 4.1), users are more inclined to interact with items that are not only in the same semantic category but are also popular within that category. Therefore, for each category, we mine multiple representative popular items that serve as proxy items.

For each category $c \in \{1, \dots, C\}$, we consider the subset $\mathcal{V}_T^c$ of attack domain items predicted to belong to category c , and compute the update magnitude of each item embedding:

$$\Delta_j = \left\| \mathbf{v}_j^{(t)} - \mathbf{v}_j^{(0)} \right\|_2, \quad v_j \in \mathcal{V}_T^c, \quad (23)$$

where $\mathbf{v}_j^{(0)}$ and $\mathbf{v}_j^{(t)}$ denote the initial and current embeddings of item $\mathbf{v}_j$, respectively. A larger Δ_j indicates stronger interaction activity and popularity.

We then select the top- N_{pop} items with the highest update magnitudes as the proxy items for category c :

$$\mathcal{V}_{\text{proxy}}^c = \{v_j \mid v_j \in \mathcal{V}_T^c, \text{rank}(\Delta_j) \leq N_{\text{pop}}\}. \quad (24)$$

These proxy items collectively represent the user-preferred interests within the category.

4.3.2 Proxy-guided Poisoned Gradient Generation. Given a target item $v^{\text{tar}} \in \mathcal{V}^{\text{tar}}$ with its category c , we generate its gradient by aligning its embedding toward the weighted centroid of the proxy items in that category:

$$\bar{\mathbf{v}}_c = \sum_{j \in \mathcal{V}_{\text{proxy}}^c} \omega_j \mathbf{v}_j, \quad \nabla \mathbf{v}^{\text{tar}} = \bar{\mathbf{v}}_c - \mathbf{v}^{\text{tar}}, \quad (25)$$

where $\omega_j = \frac{\exp(\Delta_j)}{\sum_{k \in \mathcal{V}_{\text{proxy}}^c} \exp(\Delta_k)}$ is the popularity weight of each proxy item.

Such alignment lets the target item inherit the preference bias from category-consistent popular proxies. As a result, the target item tends to achieve higher recommendation scores for users who favor this category, while exerting relatively small impact on non-target users. This design can realize precise target-user promotion, balancing effectiveness and stealthiness.

Finally, the poisoned gradient matrix uploaded by the malicious client is calculated as:

$$\nabla \tilde{\mathbf{v}}^t[i] = \begin{cases} \nabla \bar{\mathbf{v}}_i, & \text{if } v_i \in \mathcal{V}^{\text{tar}}, \\ 0, & \text{otherwise.} \end{cases} \quad (26)$$

4.4 Algorithm Analysis

In this section, we analyze ProitTUA in terms of algorithmic complexity and communication efficiency, aiming to provide a comprehensive understanding of its practicality.

4.4.1 Algorithm Complexity. ProitTUA proceeds in two stages, each incurring additional computation only on malicious clients—this design avoids imposing extra burdens on the federated server or benign clients, which is critical for practical large-scale deployment. In the following, we analyze the algorithmic complexity of each stage, and further discuss its computational cost and scalability to diverse environments.

Algorithm 1: ProitTUA on Federated Recommendation System

Input: Learning rate lr ; Benign client set $\mathcal{U}$; Malicious client set $\tilde{\mathcal{U}}$; Target item set $\mathcal{V}^{\text{tar}}$

Output: Public parameters $\mathbf{V}$ and Θ , local user embeddings $\{\mathbf{u}_i\}_{i \in \mathcal{U}}$

```

1 Initialize public parameters  $\mathbf{V}^0, \Theta^0$ , and local private  $\mathbf{u}_i^0$  for all  $u_i \in \mathcal{U}$ 
2 for each epoch  $t = 1, 2, \dots$  do
3   if  $t$  is the epoch when the attack starts,  $\mathcal{U} = \mathcal{U} \cup \tilde{\mathcal{U}}$ 
4   Sample active clients  $\mathcal{U}_t$  from  $\mathcal{U}$ 
5   for each client  $u_i \in \mathcal{U}_t$  in parallel do
6     if  $u_i \in \tilde{\mathcal{U}}$  then
7       // run on malicious client
8       // Stage 1: Item Latent Category Inference
9       auxiliary domain modeling with E.q. 14
10      fine-tune matcher using E.q. 17
11      fine-tune classifier using E.q. 20
12      infer item categories with E.q. 22
13      // Stage 2: Proxy-item based Attack Construction
14      mine proxy items for each category with E.q. 24
15       $\nabla \tilde{\mathbf{V}}^{\text{tar}} \leftarrow$  calculate target item's poisoned gradient using E.q. 25
16      upload poisoned gradients  $\nabla \tilde{\mathbf{V}}^t$  with E.q. 26 to the central server
17    else
18      // run on benign client
19      compute gradients:  $\nabla \mathbf{u}_i^t, \nabla \mathbf{V}_i^t, \nabla \Theta_i^t$  using E.q. 1
20       $\nabla \mathbf{u}_i^t \leftarrow$  update local private parameters using E.q. 2
21      upload  $\nabla \mathbf{V}_i^t, \nabla \Theta_i^t$  to the central server
22    end
23  end
24  // Server aggregation
25  Aggregate all client updates:  $\mathbf{V}^{t+1}, \Theta^{t+1} \leftarrow \text{Aggregate}(\{\nabla \mathbf{V}_i^t, \nabla \Theta_i^t\}_{i \in \mathcal{U}_t})$ 
26 end

```

Stage 1: Item Latent Category Inference. In this stage, ProitTUA mainly performs computations mainly involving two steps. In the auxiliary domain modeling step, the computations involving the recommendation model, classifier, matcher, and domain space alignment. The recommendation model's training complexity is approximately $O(|\mathcal{D}_A| \cdot B)$, where $|\mathcal{D}_A|$ is the dataset size and B the per-sample cost depending on embedding dimension d and model specifics. The classifier costs $O(|\mathcal{V}_A| \cdot (d^2 + dC))$ for all auxiliary items. The matcher complexity is upper bounded by $O(|\mathcal{V}_A|^2 \cdot (d^2 + d))$. Domain space alignment involves global alignment of complexity $O(|\mathcal{V}_A| \cdot |\mathcal{V}_T| \cdot d)$ and category subspace alignment with complexity $O(|\mathcal{V}_T^{\text{seen}}| \cdot |\mathcal{V}_A| \cdot d)$. In the attack domain adaptation step, computations mainly involving fine-tunes the matcher and classifier. The matcher fine-tuning involves $|\mathcal{V}_T^{\text{seen}}| \cdot |\mathcal{V}_T^{\text{seen}} + \mathcal{V}_A|$ item pairs, each pair requiring complexity $O(d^2 + d)$. Hence, the total complexity for matcher fine-tuning is $O(|\mathcal{V}^{\text{tar}}| \cdot |\mathcal{V}_A| \cdot (d^2 + d))$. For the classifier, pseudo labels are constructed using the fine-tuned matcher with a per-pair complexity of $O(d^2 + d)$, and the classifier incurs a pair-item forward pass cost of $O(2d^2 + 2dC)$. Thus, for $|\mathcal{V}_T^{\text{seen}}| \cdot |\mathcal{V}_T|$ item pairs, the time complexity of the classifier fine-tuning is upper bounded by $O(|\mathcal{V}_T^{\text{seen}}| \cdot |\mathcal{V}_T| \cdot (3d^2 + 2dC + d))$.

Notably, the computational cost of Stage 1 is manageable even for large-scale datasets: the core operations (fine-tuning, alignment) are lightweight and localized to malicious clients, with no cross-client data interaction or global computation.

Stage 2: Proxy-item based Attack Construction. In this stage, popularity scores are computed via ℓ_2 distance between initial and current embeddings with complexity $O(|\mathcal{V}_T| \cdot d)$. Poisoned gradients are constructed for each target item by aligning its embedding with the N_{pop} proxy items from the same class: per-target item cost $O(N_{pop} \cdot d)$. The total complexity of this stage is $O(|\mathcal{V}_T| \cdot d + |\mathcal{V}^{\text{tar}}| \cdot N_{pop} \cdot d)$. This results in negligible additional cost per malicious client—even as the federated system scales (i.e., increasing total clients, users, or items), the per-client computation remains constant, ensuring scalability for large-scale deployment.

All computations of ProitTUA are performed exclusively on a small number (e.g., 0.5%) of malicious clients. The server-side aggregation and benign clients' routines remain unchanged, meaning the overall federated system's computational efficiency is not compromised. This localized computation design, combined with the linear complexity of core operations, ensures ProitTUA maintains stable computational cost across diverse environments and scales seamlessly to large-scale federated recommendation systems.

4.4.2 Communication Efficiency. ProitTUA is designed to be compatible with standard federated training protocols and does not alter the communication pipeline. All communication between clients and server remains unchanged—each client uploads gradients of public parameters, and downloads the updated global model in return. Therefore, ProitTUA maintains the same communication cost per round as standard FedRec systems without introducing any additional communication steps or client-server coordination.

5 Experiment

We first details our experimental setup, including the datasets, baselines, defense approaches, evaluation protocol, and parameter settings. Then we report and analyze the experimental results to answer the following research questions.

- **RQ1:** Does ProitTUA effectively promote target items to their target users while reducing its impact on untargeted users?
- **RQ2:** Can ProitTUA remain stealthy by preserving the federated recommendation quality and evading existing defenses?
- **RQ3:** Does ProitTUA achieve higher attack efficiency compared to existing attacks?
- **RQ4:** What are the individual contributions of each component in each stage of ProitTUA?
- **RQ5:** RQ4: How robust is our auxiliary domain selection hypothesis?
- **RQ6:** How well do the mined proxy items reflect both popularity and category relevance?
- **RQ7:** How robust is ProitTUA across different hyperparameter settings, recommendation models, auxiliary domain sizes, and larger-scale datasets?
- **RQ8:** How effective is the proposed attack in a real-case scenario?

5.1 Experiment Settings

5.1.1 Datasets. To evaluate the effectiveness and stealthiness of ProitTUA, we conduct experiments on six widely-used real-world datasets spanning three recommendation contents: movie, book, and game. For each content, We select the dataset with a larger volume as the attack domain and the one with a smaller volume as the auxiliary domain. Detailed dataset statistics are summarized in Table 3. Notably, we select the auxiliary domain from the same recommendation task as the attack domain (e.g., another movie dataset for movie recommendation). This is because users exhibit similar preference distributions and item categories share consistent category structures

Table 3. Characteristics of different datasets across various data types. The table shows details such as domain, number of users, items, interactions, and sparsity.

DataType	Datasets	Domain	#User	#Item	#Interaction	Sparsity
Movie	MovieLens-1M	Attack	6040	3706	1000208	95.53%
	IMDB Movie	Auxiliary	2113	10109	855598	95.99%
Book	Amazon Book	Attack	14293	15017	221796	99.89%
	Book-Crossing	Auxiliary	4997	6952	33954	99.90%
Game	Steam-800k	Attack	9577	17076	801832	99.51%
	Steam-200k	Auxiliary	2389	6461	203847	98.68%

across different datasets within the same task. Such a setting ensures a relatively small domain gap and enables effective knowledge transfer for item category inference.

In the movie recommendation task, the attack domain is MovieLens-1M, which includes 6,040 users, 3,706 items, and 1,000,208 interactions. The auxiliary domain is IMDB Movie, containing 2,113 users, 10,109 items, whose items have 20 categories such as Action, Comedy, and Drama. In the book recommendation task, the attack domain is Amazon Book, comprising 14,293 users, 15,017 items, and 221,796 interactions. The auxiliary dataset is Book-Crossing, which includes 4,997 users, 6,952 items, whose items have 22 categories including Science Fiction, Biography, and Mystery. In the game recommendation task, the attack domain is Steam-800k, with 9,577 users, 17,076 items, and 801,832 interactions. The auxiliary domain is Steam-200k, containing 2,389 users, 6,461 items, whose items have 20 categories such as Role-Playing, Strategy, and Simulation. Following prior works [55, 57], for each dataset, we binarize all user-item ratings into implicit feedback and retain only users with at least five interactions. The remaining data is split into training and test sets in a 4:1 ratio based on interaction timestamps.

5.1.2 Baselines. We compare ProitTUA with representative baseline attacks, which fall into two main categories: user embedding approximation-based attacks and item popularity enhancement-based attacks.

- NoAttack: The normal federated recommendation process without any adversarial attack.
- A-ra [37]: Approximates user embeddings by sampling from a Gaussian distribution and maximizes the target item's predicted scores on these sampled embeddings.
- A-hum [37]: A variant of A-ra that uses hard user mining to better approximate user embedding distribution before maximizing the target item's predicted scores.
- PIECKUEA [62]: Approximates user embeddings based on popular items' embedding distribution and then maximizes the target item's predicted scores on these approximations.
- PSMU [57]: Generates synthetic user interactions on malicious clients, trains simulated user embeddings based on these interactions to approximate real user embeddings, and maximizes the target item's predicted scores.
- StairClimbing [19]: It first trains user embeddings on malicious client data, then constructs a Gaussian distribution from these embeddings to sample simulated user embeddings as approximations, finally maximizing the target item's predicted scores.
- PoisonFRS [55]: It identifies globally popular items by measuring similarity between each item embedding and the average embedding of all items, then promotes the target item by aligning its embedding with those popular items.

- PIECKIPE [62]: Different from PoisonFRS, the method tracks embedding updates across communication rounds to identify popular items, then enhances the target item's popularity by aligning its embedding with these identified items.

5.1.3 Defenses. We evaluate ProitTUA against both general federated learning defenses and defenses specific to federated recommender systems. The defenses considered are as follows:

- NormBound [44]: which clips the norm of each client's model update to a preset threshold to prevent any single malicious update from dominating the global model.
- Trimmed Mean [54] is a coordinate-wise robust aggregation rule. For each dimension, the server discards a fixed number of the largest and smallest values from the collected client updates, and then computes the average over the remaining values to generate the global model update.
- HiCS [57]: this approach maintains a gradient bank to accumulate client model updates over rounds. In each global round, it performs gradient sparsification by identifying and removing the largest elements of each parameter dimension from the bank. The remaining gradients are then adaptively clipped according to their average magnitude, and finally, their average is used to update the global model.
- CrossEU [13]: which analyzes variations in item gradient lists across communication rounds to detect anomalous clients, subsequently removing the most anomalous client gradients during server aggregation.
- Regular [62]: which introduces two regularization terms during client training: one encourages confusion between popular and non-popular item features to prevent malicious clients from accurately capturing popular item features and forging target items as popular; the other enforces clear separation between embeddings of popular items and users, making it difficult to infer user embeddings from popular item embeddings.
- MeAda [42]. This defense combines two adaptive strategies: *Dynamic Clipping*, which adjusts the clipping threshold each round based on the l_2 -norms of client updates, and *Adaptive Noising*, which adds Gaussian noise determined by the current threshold to satisfy differential privacy.

5.1.4 Evaluation Protocol. We evaluate each method from two perspectives: attack performance and recommendation quality. All evaluations are performed in the top-K recommendation setting.

We first define the composition of the target user set. For each target item v^{tar} , we define its target user set $\mathcal{U}_v^{\text{tar}}$ as the users who exhibit a strong preference for the category of v^{tar} . Specifically, a user u is included in $\mathcal{U}_v^{\text{tar}}$ if they have interacted with more than 10 items from the same category as v^{tar} . This heuristic reflects a practical attack scenario, where the attacker aims to influence users with genuine interest in the target item's category.

(1) Attack Performance Metrics. We evaluate how well an attack promotes target items to intended users while limiting exposure to others using four metrics averaged over all target items:

Attack Precision (AP@K): The fraction of users exposed to a target item who belong to its target user group. Higher AP@K indicates better targeting accuracy.

$$\text{AP@K} = \frac{1}{|\mathcal{V}^{\text{tar}}|} \sum_{v^{\text{tar}}} \frac{\sum_{u \in \mathcal{U}} \mathbb{I}(v^{\text{tar}} \in \mathcal{V}_u^{\text{rec}}) \cdot \mathbb{I}(u \in \mathcal{U}_v^{\text{tar}})}{\sum_{u \in \mathcal{U}} \mathbb{I}(v^{\text{tar}} \in \mathcal{V}_u^{\text{rec}})} \quad (27)$$

Attack Cover Rate (ACR@K): The fraction of all users who receive a given target item in their top-K recommendation list. Lower values indicate less exposure scope and better stealth.

$$\text{ACR@K} = \frac{1}{|\mathcal{V}^{\text{tar}}|} \sum_{v^{\text{tar}}} \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbb{I}(v^{\text{tar}} \in \mathcal{V}_u^{\text{rec}}) \quad (28)$$

Attack Error Rate (AER@K): The fraction of non-target users exposed to a target item. Lower AER@K indicates better control over non-target exposure.

$$\text{AER@K} = \frac{1}{|\mathcal{V}^{\text{tar}}|} \sum_{v^{\text{tar}}} \frac{1}{|\mathcal{U} \setminus \mathcal{U}_v^{\text{tar}}|} \sum_{u \in \mathcal{U} \setminus \mathcal{U}_v^{\text{tar}}} \mathbb{I}(v^{\text{tar}} \in \mathcal{V}_u^{\text{rec}}) \quad (29)$$

Target Cover Rate (TCR@K): To evaluate the concentration of promoted target item exposure on coherent user clusters, we define TCR@K as the fraction of target users (within the top-1 most relevant user cluster) who receive their corresponding target item in the top-K recommendations. This metric directly reflects cluster-focused attack performance, with a higher TCR@K indicating more efficient concentration of promoted target items on the intended user cluster.

$$\text{TCR@K} = \frac{1}{|\mathcal{V}^{\text{tar}}|} \sum_{v^{\text{tar}}} \frac{1}{|\mathcal{U}_v^{\text{tar}}|} \sum_{u \in \mathcal{U}_v^{\text{tar}}} \mathbb{I}(v^{\text{tar}} \in \mathcal{V}_u^{\text{rec}}) \quad (30)$$

Here, $\mathcal{V}^{\text{tar}}$ is the target item set, $\mathcal{U}$ the user set, $\mathcal{U}_v^{\text{tar}}$ the target user set for item v^{tar} , and $\mathcal{V}_u^{\text{rec}}$ the top-K recommendation list for user u . The indicator function $\mathbb{I}(\cdot)$ equals 1 if the condition holds, else 0.

(2) Recommendation Quality Metrics. To assess the performance of the federated recommendation model, we adopt three standard top-K metrics: Hit Ratio (HR@K), Normalized Discounted Cumulative Gain (NDCG@K), and Mean Reciprocal Rank (MRR@K). These metrics are computed following standard definitions in the literature [18].

5.1.5 Parameter Settings. Following [19, 57, 62], we adopt NCF [20] as backbone recommendation model. The user and item embedding dimensions are set to 32, and the prediction layer consists of hidden layers with sizes [64, 32, 16]. Both the classifier and matcher have hidden layers of sizes [64, 128]. Models are optimized using Adam [25] with a learning rate of 0.001. The total number of global communication epochs is 20. Following prior works [37, 41, 55, 57] to ensure experimental comparability, all methods initiate the attack at the 8th global communication round. After initiation, malicious clients execute the attack in every subsequent communication round until the end of training—specifically, the attack is implemented as a continuous per-round process rather than being applied intermittently at intervals of several rounds. We set $\delta = 5$ for the KL divergence margin used in classifier fine-tuning. During auxiliary domain modeling, the loss weights are set to $\lambda_{\text{cla}} = 0.5$, $\lambda_{\text{mat}} = 0.5$, and $\lambda_{\text{align}} = 0.1$. By default, we set the malicious client ratio to 0.5%, the number of target items to 4, the number of popular items per class to $N_{\text{pop}} = 4$. Specifically, the impact of these default hyperparameters will be analyzed in the experiments. All methods were implemented using the PyTorch framework (v1.8.1+cu11) and evaluated on Tesla V100 GPUs and Intel(R) Xeon(R) Platinum 8268 CPUs @ 2.90GHz.

5.2 Experimental Results

5.2.1 Attack Effectiveness (RQ1). We evaluate the attack effectiveness of ProitTUA from four perspectives: attack performance, the rank of target item, cluster-based concentration analysis and visualization before and after poisoning.

(1) Attack Performance. We evaluate the effectiveness of ProitTUA in terms of its ability to promote target items to the target users. In accordance with a fair comparison protocol, we report all metrics at the point where each method attains its maximum Target Cover Rate (i.e., $\text{TCR@10} = 100\%$). Table 4 presents the attack performance of ProitTUA alongside several baseline methods on three federated recommendation datasets: MovieLens, Amazon Book, and Steam. The evaluation metrics include AP@10 , AER@10 , and ACR@10 .

From the results, we can find that ProitTUA consistently achieves the highest attack precision (AP@10) across all datasets. For example, on the MovieLens dataset, ProitTUA attains a precision of 77.32%, outperforming the best baseline, StairClimbing (56.16%), and significantly exceeding PIECKUEA (45.48%). This indicates that most users exposed to the target items by ProitTUA are indeed target users. In addition, ProitTUA achieves the lowest attack error rate (AER@10). On the Amazon Book dataset, its error rate is as low as 15.62%, compared to 42.87% for StairClimbing and 88.36% for PIECKUEA, suggesting minimal impact on non-target users. Regarding the attack cover rate (ACR@10), ProitTUA again yields the lowest system-wide influence. For instance, on the Steam dataset, ProitTUA reduces ACR@10 to 39.37%, while StairClimbing reaches 64.72%. Moreover, although ProitTUA, PoisonFRS, and PIECKIPE all improve the target item's popularity by leveraging popular items, ProitTUA consistently outperforms the other two methods across various metrics. This is because PoisonFRS and PIECKIPE mine globally popular items, whereas ProitTUA selects locally popular items that belong to the same category as the target item. As a result, ProitTUA achieves more precise attacks on target users while causing less disturbance to non-target users.

(2) The Rank of Target Item. To further illustrate the effectiveness of ProitTUA, we visualize the average ranks of target items across training epochs, as shown in Fig. 3. We separately report the ranking dynamics for target users and non-target users on the MovieLens and Steam datasets. The results show that for target users, all methods eventually reduce the average rank of target items to below 10, indicating successful promotion of these items into the users' top-10 recommendation lists. In contrast, for non-target users, ProitTUA demonstrates a clear advantage in suppressing unintended exposure. Its final average rank of target items remains above 10, while both PSMU and StairClimbing reduce the rank below this threshold, causing target items to appear in non-target users' recommendations. These observations confirm that ProitTUA not only achieves effective item promotion for target users but also reduces disruption to non-target users.

Table 4. Attack results (%) (AP@10, AER@10, ACR@10) of different methods on FedNCF. The results are shown for MovieLens, Amazon Book, and Steam datasets. The **best** (second best) results are in bold (underlined).

Attacks	MovieLens			Amazon Book			Steam		
	AP@10↑	AER@10↓	ACR@10↓	AP@10↑	AER@10↓	ACR@10↓	AP@10↑	AER@10↓	ACR@10↓
NoAttack	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
A-ra	47.61	79.65	86.65	30.37	75.27	76.92	50.55	75.39	81.37
A-hum	45.82	82.56	89.05	17.96	78.45	79.91	47.05	77.73	84.62
PSMU	55.42	51.51	66.24	32.90	75.05	77.19	51.24	75.06	79.50
PIECKUEA	45.48	88.36	82.65	11.41	98.53	99.23	35.59	99.93	99.95
PIECKIPE	51.90	76.13	83.06	15.81	92.72	93.05	45.59	95.53	96.10
PoisonFRS	49.06	74.97	75.97	33.55	74.99	75.01	51.47	74.99	77.95
StairClimbing	<u>56.16</u>	<u>42.87</u>	<u>60.51</u>	<u>55.74</u>	<u>49.48</u>	<u>53.83</u>	<u>65.52</u>	<u>55.27</u>	<u>64.72</u>
ProitTUA(Ours)	77.32	15.62	42.12	73.13	25.83	32.45	82.13	16.44	39.37

(3) Cluster-based Concentration Analysis. To further verify that the promoted exposure is concentrated on users with consistent category preferences, we conduct an additional clustering-based analysis. We first partition users into clusters based on their historical interaction records, where users in the same cluster exhibit similar category-level preference patterns. For each target item, we compute its hit rate on every user cluster and visualize the results as a heatmap, as shown in Fig. 4.

From the heatmap, we observe that each target item achieves a significantly high hit rate on its corresponding relevant cluster, while the hit rates on other clusters remain at a low level.

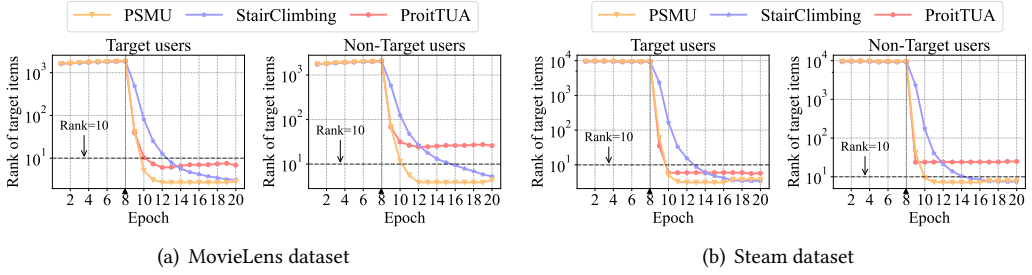


Fig. 3. Evolution of target item rankings across training epochs for target and non-target user groups under different attack methods.

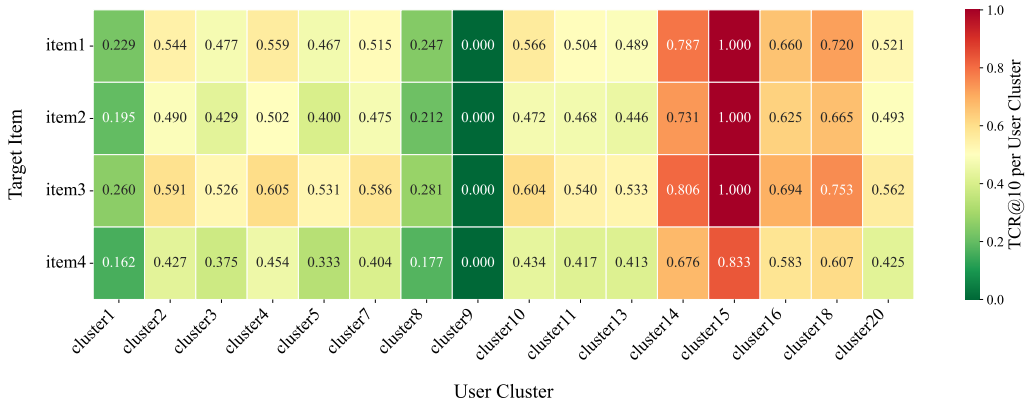


Fig. 4. Hit Rates of target items on different user clusters on MovieLens dataset.

This observation clearly demonstrates that our ProitTUA method successfully concentrates the exposure of target items on coherent user groups that share consistent category preferences, which is consistent with the goal of cluster-targeted poisoning attack.

(4) Visualization Before and After Poisoning. To further illustrate the targeted shifts of item and user representations induced by our poisoning attack, we conduct additional visualization experiments using t-SNE (t-distributed Stochastic Neighbor Embedding). Specifically, we visualize the embeddings of target items and their corresponding target user clusters (those sharing consistent category preferences with the target items) both before and after the poisoning attack, with the results presented in Fig. 5. From the t-SNE visualization, distinct shifts in embedding space are observed: before the poisoning attack, the target items are located at the edge of their corresponding target user clusters, indicating a weak correlation between target items and target users. In contrast, after the poisoning attack, the target items clearly move toward the center of the target user clusters. This phenomenon directly demonstrates that our ProitTUA attack effectively enhances the interest scores of target users toward the target items, realizing the intended targeted shift in the embedding space and verifying the effectiveness of our cluster-targeted poisoning strategy.

5.2.2 Attack Stealthiness (RQ2). This subsection evaluates the stealthiness of ProitTUA, i.e., whether it can preserve the overall recommendation performance and evading various defenses.

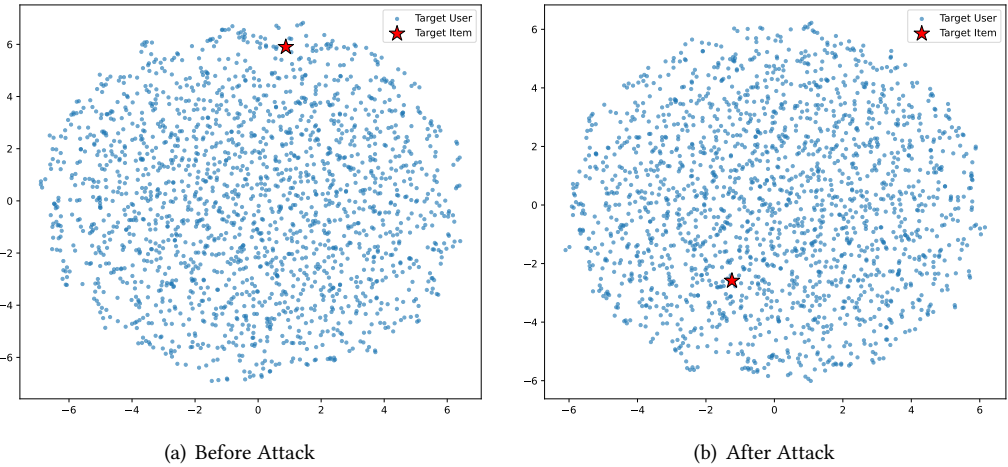


Fig. 5. t-SNE visualization of target item and its target user cluster before and after poisoning attack on MovieLens dataset.

(1) Impact on Recommendation Performance. An effective poisoning attack should preserve overall recommendation quality to ensure stealthiness [41, 57]. Table 5 summarizes the recommendation performance in terms of HR@10, NDCG@10, and MRR@10 on the MovieLens, Amazon Book, and Steam datasets under different attack methods. Among all approaches, ProitTUA causes the least degradation in recommendation quality. On MovieLens, ProitTUA achieves HR@10 = 67.05%, NDCG@10 = 16.57%, and MRR@10 = 36.93%, which are close to the NoAttack baseline values of 68.78%, 18.69%, and 37.03%, respectively. Similarly, on the Steam dataset, ProitTUA attains HR@10 = 7.36% and NDCG@10 = 0.83%, close to the NoAttack scores of 7.93% and 0.94%. This indicates that ProitTUA is capable of maintaining strong recommendation performance while executing the attack. In contrast, other attack methods lead to more severe degradation. For example, PIECKIPE reduces the MRR@10 on MovieLens to 17.20%, which is less than half of the NoAttack baseline, clearly undermining recommendation quality. These results demonstrate that ProitTUA can maintain stable recommendation quality, which contributes to improved stealthiness of the attack.

Table 5. Recommendation results (%) (HR@10, NDCG@10, MRR@10) of different methods on FedNCF. The results are shown for MovieLens, Amazon Book, and Steam datasets.

Methods	MovieLens			Amazon Book			Steam		
	HR@10↑	NDCG@10↑	MRR@10↑	HR@10↑	NDCG@10↑	MRR@10↑	HR@10↑	NDCG@10↑	MRR@10↑
NoAttack	68.78	18.69	37.03	0.443	0.063	0.095	7.93	0.94	2.97
A-ra	59.58	11.15	24.48	0.239	0.036	0.055	5.77	0.65	1.99
A-hum	59.27	11.11	23.94	0.235	0.034	0.054	5.36	0.62	1.94
PSMU	62.33	15.14	35.03	0.260	0.037	0.058	5.97	0.67	2.04
PIECKUEA	53.65	7.37	10.44	0.133	0.014	0.015	5.34	0.38	0.67
PIECKIPE	61.55	11.18	17.20	0.136	0.017	0.017	5.59	0.44	0.86
PoisonFRS	61.57	13.70	30.66	0.272	0.037	0.037	5.81	0.46	0.95
StairClimbing	63.74	15.53	36.12	0.274	0.038	0.059	6.37	0.70	2.10
ProitTUA(Ours)	<u>67.05</u>	<u>16.57</u>	<u>36.93</u>	<u>0.422</u>	<u>0.057</u>	<u>0.088</u>	<u>7.36</u>	<u>0.83</u>	<u>2.68</u>

(2) Circumventing Various Defenses. To evaluate the robustness of ProitTUA against existing defense mechanisms, we compare its performance with two strong baselines, PSMU and StairClimbing, under six widely-used defenses: NormBound, CrossEU, Regularization, TrimmedMean, HiCS and MeAda. Notably, all experiments are conducted under the same attack budget constraint—specifically, the proportion of malicious clients required for each method to achieve 100% TCR@10, ensuring a fair comparison of stealthiness and robustness under equivalent target effectiveness. Table 6 summarizes attack and recommendation metrics on the MovieLens dataset. We use ACR@10 and TCR@10 to assess attack effectiveness under defense, where ACR@10 reflects the attack’s overall impact on all users and TCR@10 measures its success on the target user group.

Two key observations emerge from the results. First, ProitTUA demonstrates strong resilience to these defenses, maintaining substantial attack capability even under stringent protection. For example, under HiCS, ProitTUA achieves an ACR@10 of 28.70% and TCR@10 of 70.94%, whereas PSMU and StairClimbing’s attack effectiveness is nearly nullified (both with ACR@10 and TCR@10 close to zero or very low). This indicates that ProitTUA’s poisoned gradients are more stealthy, making them less likely to be detected or removed by defense strategies. By comparison, existing approaches tend to launch large-scale attacks, which produce more noticeable gradient anomalies that are more susceptible to suppression by defense mechanisms. Second, the deployment of effective defense mechanisms leads to an improvement in the recommendation performance of FedRecs, as reflected by the increased HR@10 and NDCG@10 scores compared to the NoDefense setting. For instance, PSMU’s HR@10 improves from 63.16 (NoDefense) to 68.15% (TrimmedMean), and StairClimbing’s HR@10 rises from 63.74% to 68.60%.

Table 6. Performance (%) of different defenses under various attack and recommendation metrics on FedNCF.

Defenses	PSMU				StairClimbing				ProitTUA(Ours)			
	Attack		Recommendation		Attack		Recommendation		Attack		Recommendation	
	ACR@10	TCR@10	HR@10	NDCG@10	ACR@10	TCR@10	HR@10	NDCG@10	ACR@10	TCR@10	HR@10	NDCG@10
NoDefense	66.24	100.0	63.16	15.14	60.51	100.0	63.74	15.53	42.12	100.0	67.05	16.57
NormBound	0.55	1.73	68.10	18.15	51.03	95.74	66.27	16.31	42.12	99.98	67.05	16.57
CrossEU	0.00	0.00	68.61	18.36	48.03	85.88	65.69	16.39	42.77	97.67	66.95	17.27
Regular	47.17	90.53	65.65	16.39	25.39	75.93	68.27	18.26	41.03	99.51	63.86	15.84
TrimmedMean	0.00	0.00	68.15	18.31	24.01	70.71	68.60	18.75	30.64	87.93	68.22	17.40
HiCS	0.00	0.00	68.12	18.16	3.88	12.28	67.83	18.57	28.70	70.94	67.00	17.18
MeAda	0.00	0.00	68.13	18.17	2.56	9.87	64.12	18.07	26.22	65.50	67.20	17.25

5.2.3 Attack Efficiency (RQ3). We further analyze the attack efficiency of ProitTUA by examining how attack performance (TCR@10) and recommendation quality (HR@10) evolve during training. We track these metrics over communication rounds to observe the dynamics of both malicious influence and overall recommendation utility. Figure 3 presents the training curves of TCR@10 and HR@10 on the MovieLens and Steam datasets, comparing ProitTUA with baseline methods.

ProitTUA achieves the best TCR@10 significantly faster than baselines. Specifically, on the MovieLens dataset, it requires only 3 epochs to reach its optimal TCR@10, while StairClimbing and PSUM take around 6 and 8 epochs, respectively. This demonstrates that ProitTUA is more efficient, which is crucial in adversarial scenarios. In contrast, existing methods perform large-scale attacks without precisely targeting the target users, which slows down their efficiency. Moreover, ProitTUA imposes less degradation on overall recommendation quality. As shown in HR@10 curves, ProitTUA maintains stable recommendation performance throughout the training process, whereas baselines cause a noticeable drop in HR@10 as their attack intensifies. This highlights ProitTUA’s ability to remain recommendation performance while conducting effective attack.

5.2.4 Ablation Study (RQ4). ProitTUA consists of four major steps grouped into two stages. We conduct an ablation study to assess the contribution of each step: (1) auxiliary domain modeling, (2) attack domain adaptation, (3) category-wise proxy item mining, and (4) proxy-guided poisoned gradient generation. Table 7 reports the attack effectiveness metrics under various ablated configurations on three datasets.

(1) Auxiliary Domain Modeling. We ablate two domain alignment strategies: GSA (Global Space Alignment) and CSA (Category Subspace Alignment). Removing either leads to a noticeable drop in precision and targeting. For instance, removing both GSA and CSA results in a significant performance drop on MovieLens (AP@10 drops from 77.32% to 69.20%), indicating that domain alignment is essential for cross-domain knowledge transfer.

(2) Attack Domain Adaptation. We evaluate the effects of removing FM (Fine-tune Matcher) and FC (Fine-tune Classifier). Both components are critical to accurate class prediction in the attack domain. Without classifier fine-tuning, AP@10 drops from 77.32% to 46.45% on MovieLens, showing that classifier fine-tuning is essential. Notably, removing both FM and FC severely degrades attack performance across all metrics, highlighting the necessity of domain adaptation.

(3) Category-wise Proxy Item Mining. To evaluate the advantage of our category-wise proxy item mining, we replace it with two unsupervised clustering-based methods—K-Means and Gaussian Mixture Model (GMM)—applied to the item embedding space. For both methods, the number of clusters is set equal to the number of item categories, and we select the most popular item within each cluster as the proxy item. However, as shown in Table 7, the attack performance under these clustering-based proxies is significantly lower than that of our proposed approach. For example, on the MovieLens dataset, AP@10 drops from 77.32% to 54.71% (K-Means) and 56.34% (GMM), respectively. This degradation stems from the lack of explicit category guidance: while clustering groups items with similar embeddings, it does not ensure semantic consistency in terms of item categories. Consequently, the mined proxy items often fail to align categorically with the target items, undermining their effectiveness in guiding poisoning gradients.

(4) Proxy-guided Poisoned Gradient Generation. To validate the importance of using category-consistent proxy items for gradient generation, we compare our proposed alignment strategy with a misaligned baseline (Mis-Align), where target items are intentionally aligned with proxy items from different categories. As expected, the Mis-Align variant exhibits drastically reduced attack performance (e.g., AP@10 drops to 25.00% on MovieLens), with significantly higher AER@10 (62.51%). This demonstrates that category consistency between target items and proxy items is critical for effective and accurate target-user promotion. Only by aligning with category-consistent proxies can the attack inherit proper preference bias and achieve selective exposure to target users.

5.3 Visualization Analysis for Auxiliary Domain Robustness Verification (RQ5)

To verify the robustness of our auxiliary domain selection hypothesis, we conduct t-SNE visualization on the feature distributions of comedy movies under the movie recommendation task, which serves as a typical case for analysis. We first acknowledge that domain discrepancy is inevitable between the auxiliary domain and the attack domain. As shown in the Figure ??, without additional alignment operations, the feature distributions of comedy items from the two domains form obvious and clear separation boundaries during model training, revealing inherent domain gaps even under the same recommendation task.

To alleviate such domain discrepancy and further enhance the rationality of auxiliary domain selection, we adopt the proposed domain space alignment strategy. After implementing this alignment operation, the feature distributions of comedy movies from the auxiliary domain and the attack domain gradually converge and tend to be consistent in the latent embedding space. This

Table 7. Results (%) of attack metrics (AP@10, TCR@10, AER@10, ACR@10) in ablation experiments of different stages across various datasets.

Variants	MovieLens				Amazon Book				Steam			
	AP@10↑	TCR@10↑	AER@10↓	ACR@10↓	AP@10↑	TCR@10↑	AER@10↓	ACR@10↓	AP@10↑	TCR@10↑	AER@10↓	ACR@10↓
Auxiliary Domain Modeling												
w/o GSA	70.15	94.09	20.89	46.02	67.75	96.73	33.45	36.28	72.82	87.74	38.42	43.82
w/o CSA	72.14	96.70	18.42	44.22	69.64	99.60	31.32	35.20	75.72	92.67	36.38	42.96
w/o GSA & CSA	69.20	82.70	30.67	49.22	64.64	90.60	36.32	37.20	60.72	80.67	41.38	45.20
Attack Domain Adaptation												
w/o FC	46.45	78.49	37.45	49.24	52.71	78.68	46.36	49.23	55.74	67.70	42.37	44.19
w/o FM	58.78	80.87	33.46	40.25	60.70	82.67	40.35	43.22	61.73	72.69	39.39	43.20
w/o FC & FM	41.86	70.84	50.56	56.31	48.76	75.74	52.46	54.29	50.83	61.75	46.43	45.26
Category-wise Proxy Item Mining												
K-Means	67.34	65.30	43.96	50.70	45.64	71.54	62.34	68.42	22.56	74.88	74.98	74.95
GMM	70.50	70.43	35.79	46.73	42.13	73.63	66.59	70.24	38.05	51.39	50.04	50.49
Proxy-guided Poisoned Gradient Generation												
Mis-Align	25.00	22.51	62.51	45.27	32.13	45.63	65.39	58.24	30.72	28.68	60.38	49.22
ProitTUA	77.32	100.0	15.62	42.12	73.13	100.0	25.73	32.45	82.13	100.0	16.44	39.37

observation fully validates the rationality and robustness of our auxiliary domain selection hypothesis, and further demonstrates three key conclusions: (1) The designed domain alignment method effectively narrows the domain gap between the auxiliary and attack domains; (2) The item feature distributions of the two domains become sufficiently close after alignment, which lays a solid foundation for the subsequent fine-tuning of the category classifier pre-trained on the auxiliary domain in the attack domain; (3) The consistent category structure across domains is well preserved, providing reliable semantic support for the follow-up proxy item mining in the attack process.

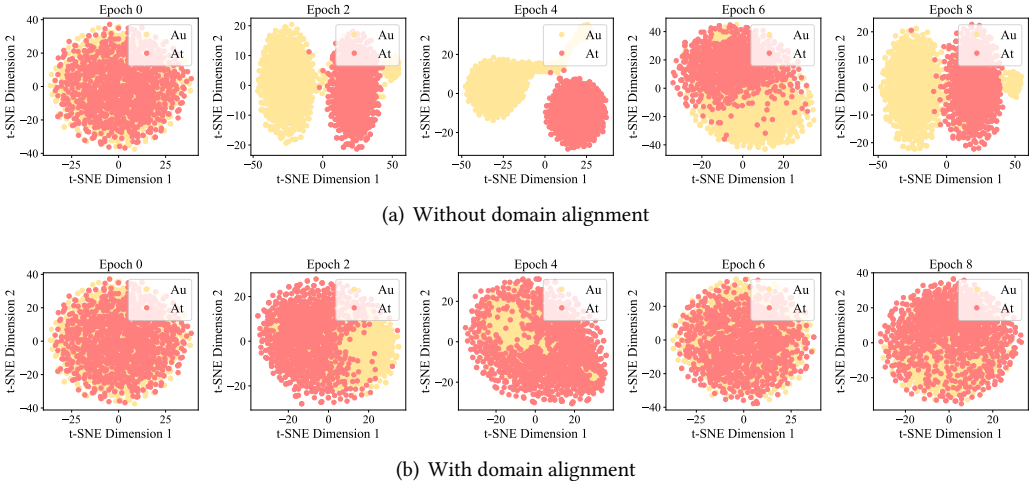


Fig. 6. t-SNE visualization of comedy movie feature distributions between the auxiliary domain (au) and attack domain (at). (a) depicts obvious domain gaps and clear separation boundaries without alignment operations; (b) shows convergent and consistent feature distributions after domain space alignment.

5.3.1 Quality of Proxy Items (RQ6). To assess the quality of the mined proxy items, we conduct a visual analysis from two perspectives—*popularity* and *category relevance*—across three datasets: comedy movies from MovieLens, reference books from Amazon, and horror games from Steam (see Figure 7).

The visualizations reveal that the mined proxy items consistently exhibit high popularity, typically ranking within the top 50% of items in their respective datasets. For instance, in the Amazon Book domain, the identified proxy items include references with popularity ranks of 9, 12, 24, 26, and 28, indicating their strong presence in user interactions. Furthermore, a majority of the mined items are correctly categorized, as reflected by the dominance of green points in the plots, which denote correct category assignments. These findings demonstrate that our proxy item mining strategy is both effective and precise in selecting high-popularity, category-wise proxy items across different domains.

To explain why the mined proxy items exhibit high category relevance, we report the category inference accuracy of popular items. Specifically, we evaluate a subset of 100 popular items, denoted as $\mathcal{V}_{T, \text{pop}}^{\text{unseen}}$, sampled from the MovieLens, Amazon Book, and Steam datasets. As shown in Table 8, the accuracy on the popular subset $\mathcal{V}_{T, \text{pop}}^{\text{unseen}}$ is relatively high—for example, reaching 65.12% on the MovieLens dataset. This improvement can be attributed to the richer interaction histories associated with popular items, which lead to more semantically informative embeddings and thereby facilitate more accurate category prediction.

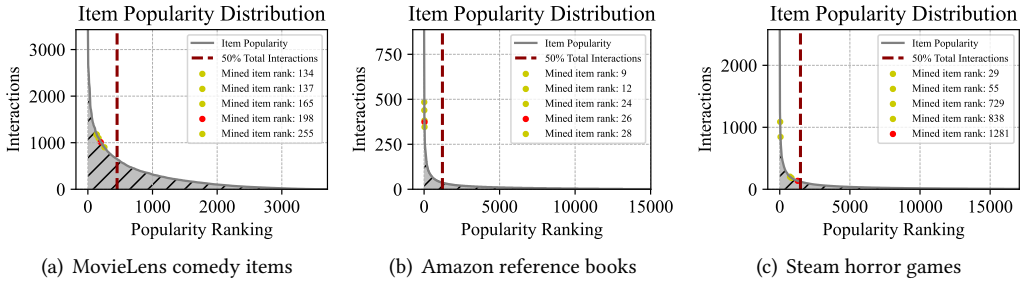


Fig. 7. Visualization of popular items mined for different semantic classes across datasets. Green points denote correctly classified items, while red points indicate misclassified items.

Table 8. Category inference accuracy (%) of popular items across three datasets.

Item Subset	MovieLens	Amazon Book	Steam
$\mathcal{V}_{T, \text{pop}}^{\text{unseen}}$	65.12%	58.13%	60.18%

5.3.2 Robustness Evaluation (RQ7). We evaluate the robustness of ProitTUA from four perspectives: (1) its sensitivity to key hyperparameters, (2) its generalization to the base federated recommendation model, (3) its adaptability to variations in auxiliary domain size, and (4) its generalization to larger-scale datasets.

(1) Sensitivity to Hyperparameters. We investigate how the performance of ProitTUA varies with respect to several critical hyperparameters: the number of mined popular items per class (N_{pop}), the proportion of malicious clients, the number of target items, and the proportion of items with category labels held by malicious clients.

Number of Mined Popular Items. Figure 8 shows the attack performance of ProitTUA under different values of $N_{\text{pop}} \in \{1, 2, 4, 6, 8\}$ on FedNCF. A consistent trend is observed across both datasets: performance initially improves and then declines, with the best results achieved when $N_{\text{pop}} = 4$. This observation aligns with our design intuition. Using too few popular items (e.g.,

$N_{pop} = 1$ or 2) fails to capture sufficient semantic diversity within each class, leading to suboptimal alignment with target user preferences. On the other hand, incorporating too many items introduces noise from semantically unrelated content, which degrades the consistency of category-level embedding alignment.

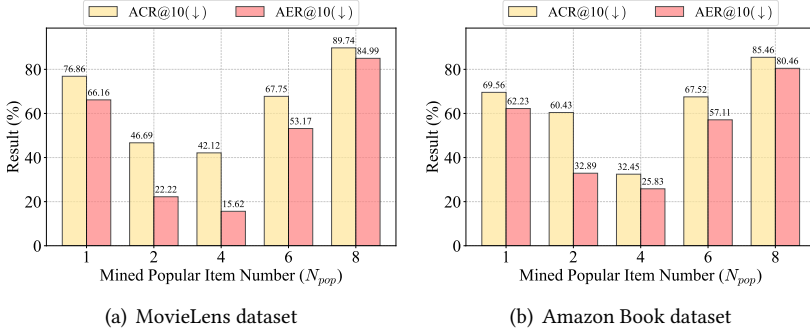


Fig. 8. Effects of hyper-parameter mined popular item number (N_{pop}).

Proportion of Malicious Clients. We evaluate attack efficiency under varying proportions of malicious clients: 0.1%, 0.2%, 0.5%, 1.0%, 2.0%. The number of target items is all 4. For each setting, we report the number of global training rounds required by each method to achieve full target user coverage (i.e., $TCR@10 = 100\%$) on FedNCF. As shown in Figure 9, at low malicious client proportions (e.g., 0.1%), ProitTUA appears to be the only method capable of successfully reaching some target users, while most baselines struggle to launch effective attacks, as indicated by the ✕ marks. As the proportion of malicious clients increases, all methods show improved efficiency, and ProitTUA consistently requires fewer epochs to achieve full target user coverage. For example, on the MovieLens dataset, as the malicious client ratio increases from 0.2% to 2.0%, ProitTUA reduces its convergence time from 4 to 2 rounds, compared to 7–3 rounds for PSMU and 9–2 rounds for StairClimbing. These results suggest that due to ProitTUA’s fine-grained targeting of specific user groups, it is more likely to achieve effective target item promotion with a smaller proportion of malicious clients, whereas existing methods tend to require a larger number of malicious clients or more attack rounds to achieve similar effects.

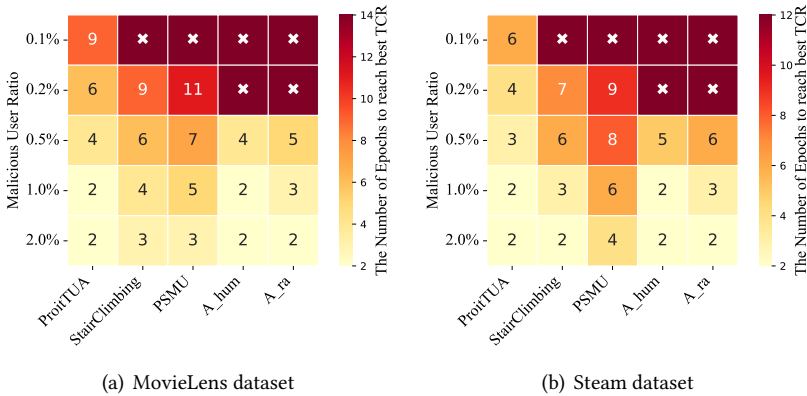


Fig. 9. Performance of different attack methods under different proportions of malicious users.

Number of Target Items. We further evaluate the scalability of ProitTUA by varying the number of target items to promote: {1, 2, 4, 6, 8}. The proportion of malicious clients is uniformly set to 0.5%. Figure 10 shows the number of global training rounds required to reach full $TCR@10$

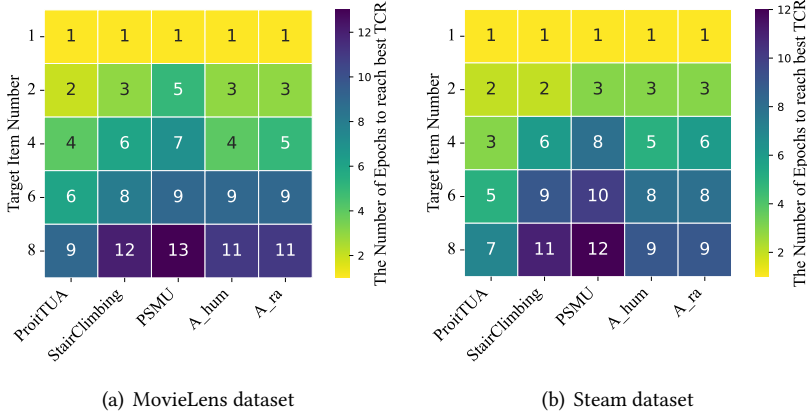


Fig. 10. Performance of different attack methods under different target item number.

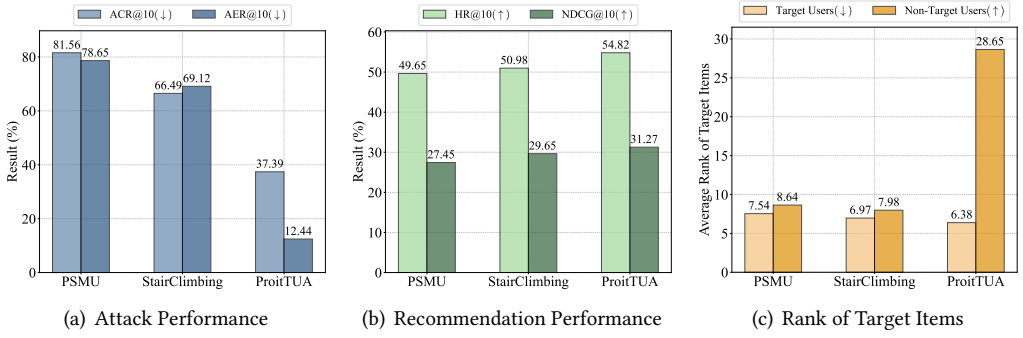


Fig. 11. Results of different methods on the FedDMF system for MovieLens.

for each method on FedNCF. Across all methods, we observe a clear upward trend—attacking more target items increases the required training epochs. Despite this, ProiTUA consistently converges faster than all baselines. For instance, when promoting 8 target items on MovieLens, ProiTUA reaches full TCR@10 in just 7 rounds, whereas StairClimbing and PSMU require more than 11 and 12 rounds, respectively. These results demonstrate that ProiTUA scales more efficiently to multi-item promotion scenarios.

(2) Effectiveness across Different Rec Models. To evaluate the generalizability of ProiTUA beyond the FedNCF backbone, we assess its performance under an alternative federated recommendation model—FedDMF[53]—on the MovieLens dataset. As shown in Figure 11, we compare ProiTUA with two strong baselines (PSMU and StairClimbing) along three dimensions: attack effectiveness, recommendation quality, and target item rank.

ProiTUA achieves the lowest ACR@10 (37.39%) and AER@10 (12.44%), demonstrating strong targeting precision by effectively focusing on intended users while minimizing exposure and disruption to non-target users. It also attains the highest HR@10 (54.82%) and NDCG@10 (31.27%), outperforming PSMU (50.38%, 27.89%) and StairClimbing (47.96%, 25.60%), thereby better preserving overall recommendation quality. Furthermore, target items are ranked significantly lower in non-target users' recommendation lists, further confirming ProiTUA's high precision and minimal collateral impact. These results indicate that ProiTUA can maintain its attack effectiveness and stealthiness across different backbone models, showcasing its robust generalizability in federated recommendation scenarios.

(3) Impact of Auxiliary Domain Size on Attack Performance. To clarify how the size of the auxiliary domain affects ProitTUA’s attack effectiveness, we conduct additional experiments on the MovieLens dataset. Specifically, we vary the number of items in the auxiliary domain (denoted as the proportion of auxiliary items relative to the full auxiliary domain) and evaluate ProitTUA’s attack performance under three typical settings: 25%, 50%, and 100% of the original auxiliary item count. We use AP@10 (Attack Performance@10, higher is better), AER@10 (Attack Error Rate@10, lower is better), and ACR@10 (Attack Corruption Rate@10, lower is better) as evaluation metrics, which are consistent with the metrics used in previous experiments to ensure comparability.

Table 9. Attack results (%) (AP@10, AER@10, ACR@10) of ProitTUA with different auxiliary domain sizes. The **best** (second best) results are in bold (underlined).

Size	MovieLens			Amazon Book			Steam		
	AP@10↑	AER@10↓	ACR@10↓	AP@10↑	AER@10↓	ACR@10↓	AP@10↑	AER@10↓	ACR@10↓
25%	40.21	49.65	62.44	42.55	76.99	82.01	42.40	76.60	86.92
50%	<u>60.21</u>	<u>29.53</u>	<u>54.45</u>	<u>60.32</u>	<u>56.41</u>	<u>59.01</u>	<u>62.58</u>	<u>60.25</u>	<u>69.75</u>
100%	77.32	15.62	42.12	73.13	25.83	32.45	82.13	16.44	39.37

As shown in Table 9, the size of the auxiliary domain significantly affects ProitTUA’s attack performance: First, ProitTUA performs poorly with only 25% auxiliary domain size, achieving an AP@10 of 40.21% and considerably high AER@10 (49.65%) and ACR@10 (62.44%) on MovieLens, due to insufficient category information for accurate inference. Second, increasing the auxiliary domain to 50% noticeably improves performance: AP@10 rises to 60.21%, with AER@10 and ACR@10 decreasing to 29.53% and 54.45%, respectively, though it still lags behind the full-size setting. Third, the 100% auxiliary domain achieves the best performance with comprehensive category information. These results indicate that a sufficiently large auxiliary domain is required to ensure effective target user attacks.

(4) Evaluation on Wider Datasets. To further evaluate ProitMTA in large-scale federated recommendation systems and assess its robustness beyond potential popularity biases, we use MovieLens-10M as the attack domain and MovieLens-1M as the auxiliary domain. MovieLens-10M contains 10,000,054 ratings from 71,567 users and 10,681 items. The experiments demonstrate that ProitMTA maintains high attack effectiveness on MovieLens-10M while using MovieLens-1M as the auxiliary source, indicating the method generalizes to larger real-world datasets. The attack performance is reported in Table 10. Experimental results show that ProitTUA consistently achieves significantly better performance than PSMU and StairClimbing on the large-scale MovieLens-10M dataset. These results confirm that the category popularity-based proxy item mining strategy does not overfit to popularity bias and generalizes effectively to large-scale real-world federated recommendation systems.

Table 10. Attack performance (%) on the large-scale MovieLens-10M dataset.

Attacks	AP@10 ↑	AER@10 ↓	ACR@10 ↓
PSMU	57.75	62.46	75.54
StairClimbing	62.45	58.13	65.18
ProitTUA	80.23	15.89	38.79

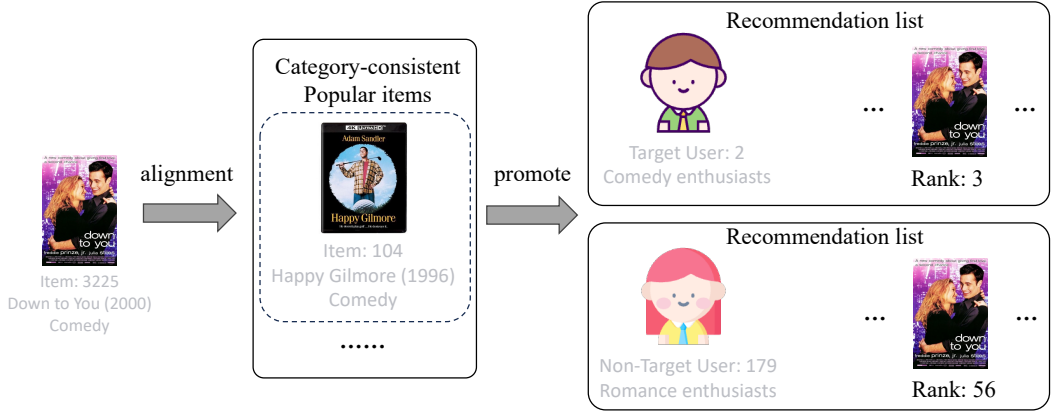


Fig. 12. Case study: exposure of one target item to target user and non-target user.

5.3.3 Case Study (RQ8). To intuitively validate the effectiveness and target specificity of ProitTUA, we present a detailed case study on real users and items. We select target item 3225: Down to You (2000), a typical comedy movie, and evaluate its recommendation ranking on two representative users: target user 2, a comedy enthusiast, and non-target user 179, a romance enthusiast.

As illustrated in Fig. 12, after launching our attack, the target comedy movie is promoted to rank 3 for the target user, standing out in the top recommendation positions. In contrast, the same item only reaches rank 56 for the non-target user with unrelated preference, showing nearly no promotion. This striking contrast confirms that ProitTUA can accurately amplify the exposure of target items to intended users while keeping minimal influence on non-target users, which well demonstrates the precise targeting capability of our attack.

6 Discussion on the Defense of ProitTUA

ProitTUA exposes the vulnerability of federated recommender systems to target user attack, while existing defenses fail to effectively mitigate such threats. To address this, we propose a novel server-side defense method grounded in an analysis of gradient change patterns during training.

Observation. We track the ℓ_2 -norm of global gradients for normal items with varying popularity (high, medium, low) across training epochs. As shown in Figure 13, these normal items with different popularity levels all follow a structured pattern: their gradient norms increase during early optimization, reach a peak in the middle phase, and gradually diminish as convergence is achieved. This reflects stable training signals driven by user interactions.

We argue that manipulated target items tend to deviate from natural training dynamics and exhibit anomalous temporal patterns. Specifically, these items often undergo abrupt transitions, resulting in nonsmooth and irregular variations—such as sudden surges in gradient magnitude upon attack launch—that are rarely observed during the training of normal items.

Step 1: Suspicious Item Detection. We begin by monitoring the gradient-norm trajectory of each item $v \in \mathcal{V}_T$ across training epochs. Specifically, the gradient-norm fluctuation at epoch i is computed as:

$$F_v^i = \left(\|\nabla \mathbf{v}^i\|_2 - \|\nabla \mathbf{v}^{i-1}\|_2 \right)^2, \quad (31)$$

where $\nabla \mathbf{v}^i$ and $\nabla \mathbf{v}^{i-1}$ denote the gradients of item v at epochs i and $i-1$, respectively.

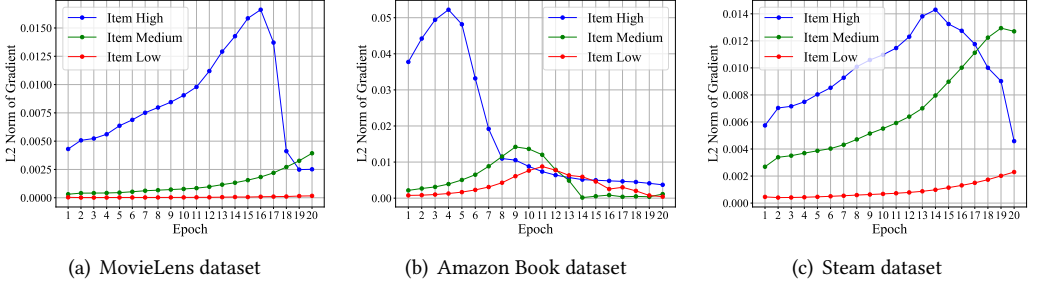


Fig. 13. Temporal evolution of ℓ_2 -norms of normal item gradients across epochs. Normal items follow a smooth and regular pattern.

To quantify the irregularity of an item's training behavior, we calculate its *suspicion score* (SUS) as the maximum gradient-norm fluctuation observed across all epochs up to the current epoch t :

$$SUS(v) = \max_{2 \leq i \leq t} F_v^i. \quad (32)$$

This score captures the most significant deviation in the item's training trajectory. Intuitively, normal items typically exhibit smooth gradient transitions due to stable local interactions, while poisoned items tend to show abrupt, irregular changes.

Then we rank all items based on their suspicion scores and select the top- τ most anomalous items to form the suspicious item set:

$$\mathcal{V}^{sus} = \text{Top-}\tau\{v \in \mathcal{V}_T \mid \text{ranked by } SUS(v)\}. \quad (33)$$

This procedure identifies suspicious items that deviate from normal training dynamics.

Step 2: Suspicious Client Tracing. After detecting suspicious items, we trace back to identify the clients likely responsible for manipulating them. For each suspicious item $v \in \mathcal{V}^{sus}$, we extract its top- ρ gradient dominant client set:

$$\mathcal{U}_v^{dom} = \text{Top-}\rho \{u \in \mathcal{U}_t \mid \text{ranked by } \|\nabla \mathbf{v}_u^t\|_2\}. \quad (34)$$

where $\nabla \mathbf{v}_u^t$ denotes the gradient of item v uploaded by client u at epoch t .

To detect suspicious clients that consistently exhibit abnormal behavior, we identify those who appear across the dominant client sets of all suspicious items:

$$\mathcal{U}^{sus} = \bigcap_{v \in \mathcal{V}^{sus}} \mathcal{U}_v^{dom}. \quad (35)$$

This intersection strategy ensures that only clients who consistently dominate the training of multiple abnormal items are flagged as suspicious, thereby reducing false positives.

Step 3: Suspicious Client Exclusion. In subsequent model aggregation steps, we completely exclude the updates from $\mathcal{U}^{sus}$, preventing their gradients from influencing the global model:

$$\mathbf{V}^{t+1} = \mathbf{V}^t - lr \sum_{u_i \in \mathcal{U}_t \setminus \mathcal{U}^{sus}} \nabla \mathbf{v}_i^t, \quad (36)$$

where $\nabla \mathbf{v}_i^t$ denotes the gradient matrix uploaded by client u_i . This strategy effectively eliminates the poisoned gradients that cause item promotion while preserving gradients from benign users.

Algorithmic Complexity. The computational complexity of the proposed defense mainly consists of three parts: Detecting suspicious items involves computing the gradient norm fluctuations for all items $\mathcal{V}_T$, which requires $O(|\mathcal{V}_T|)$ time, followed by sorting the items by their suspicion scores with a complexity of $O(|\mathcal{V}_T| \log |\mathcal{V}_T|)$. For the detected suspicious client set, sorting the gradients of contributing clients for each suspicious item takes $O(|\mathcal{U}_t| \log |\mathcal{U}_t|)$. During model

aggregation, filtering out updates from suspicious clients incurs $O(|\mathcal{U}^{sus}|)$ complexity, where $|\mathcal{U}_t|$ is the total number of clients participating in the current round. Overall, the total time complexity per epoch is $O((1 + \log |\mathcal{V}_T|) \cdot |\mathcal{V}_T| + \tau \cdot |\mathcal{U}_t| \log |\mathcal{U}_t| + |\mathcal{U}^{sus}|)$. Typically, $|\mathcal{V}_T| \gg |\mathcal{U}_t|$ and the suspicious item count τ is small, so this defense is efficient and scalable in practice.

Evaluation. To validate the effectiveness of our proposed defense against ProitTUA, we conduct experiments on two benchmark datasets: MovieLens and Steam, using $\tau = 4$ and $\rho = 5\%$ as defense parameters. Figure 14 illustrates the ranking dynamics of target items under two scenarios: *NoDefense* and *OurDefense*. For each scenario, we report the average rank positions of target items for both target users and non-target users.

The experimental results demonstrate that, in the absence of defense, ProitTUA can significantly degrade the ranking positions of target items, effectively promoting them to higher positions within the recommendation lists. This confirms the strong attack capability of ProitTUA in manipulating federated recommender systems. In contrast, when our defense mechanism is used, this abnormal promotion of rank is substantially suppressed, indicating that our defense effectively mitigates the influence of ProitTUA.

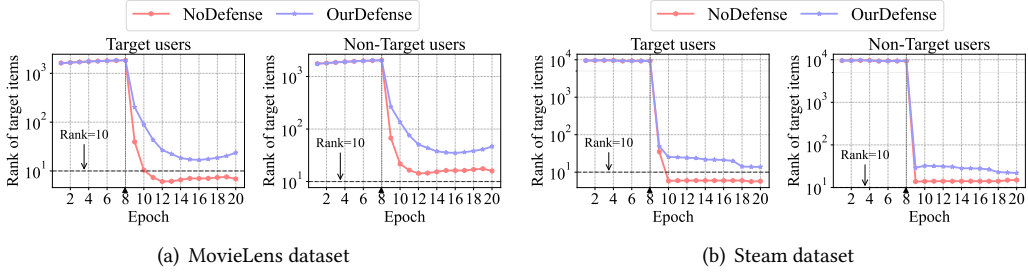


Fig. 14. Comparison of average rank of target items under ProitTUA with and without defense.

7 Conclusion

In this article, we proposed **ProitTUA**, a novel proxy-item based target user attack method for federated recommender systems (FedRecs). Unlike existing approaches that aim to promote target items to all users, ProitTUA selectively promotes each target item to a semantically relevant user group by leveraging category-wise proxy items. To enable this fine-grained promotion under limited category prior knowledge, ProitTUA adopts a two-stage framework: (1) *item latent category inference*, which leverages auxiliary domain modeling and attack domain adaptation to infer the latent categories of items under limited category prior knowledge; and (2) *proxy-item based attack construction*, which aligns each target item with representative category-level proxy items. Extensive experiments conducted on multiple datasets and FedRec frameworks demonstrate the superior effectiveness and stealthiness of our attack. Furthermore, we design a defense mechanism based on gradient dynamics analysis to mitigate the threat of ProitTUA, which shows promising effectiveness. Our work not only reveals critical security vulnerabilities in FedRecs but also motivates future research toward more robust federated recommender systems.

Acknowledgments

We gratefully acknowledge the support and sponsorship that have been crucial in enabling this research. This work is supported by the Major Key Project of PCL (Grant No. PCL2024A05). This work is supported by Guangdong Major Project of Basic and Applied Basic Research (No. 2019B030302002). This work is supported by Shenzhen Science and Technology Program (Grant No. ZDSYS20210623091809029).

References

- [1] Christopher Bret Alexander. 2019. The general data protection regulation and California consumer privacy act: The economic impact and future of data privacy regulations. *Loy. Consumer L. Rev.* 32 (2019), 199.
- [2] Muhammad Ammad-Ud-Din, Elena Ivannikova, Suleiman A Khan, Were Oyomno, Qiang Fu, Kuan Eeik Tan, and Adrian Flanagan. 2019. Federated collaborative filtering for privacy-preserving personalized recommendation system. *arXiv preprint arXiv:1901.09888* (2019).
- [3] John Ayeelyan, Sapdo Utomo, Adarsh Rouniyar, Hsiu-Chun Hsu, and Pao-Ann Hsiung. 2025. Federated learning design and functional models: Survey. *Artificial Intelligence Review* 58, 1 (2025), 1–38.
- [4] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How to backdoor federated learning. In *International conference on artificial intelligence and statistics*. PMLR, 2938–2948.
- [5] Alain Berlinet and Christine Thomas-Agnan. 2011. *Reproducing kernel Hilbert spaces in probability and statistics*. Springer Science & Business Media.
- [6] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. 2017. Machine learning with adversaries: Byzantine tolerant gradient descent. *Advances in neural information processing systems* 30 (2017).
- [7] Qiqi Cai, Jian Cao, Guandong Xu, and Nengjun Zhu. 2024. Distributed Recommendation Systems: Survey and Research Directions. *ACM Trans. Inf. Syst.* 43, 1, Article 10 (Nov. 2024), 38 pages. doi:10.1145/3694783
- [8] Renqin Cai, Jibang Wu, Aidan San, Chong Wang, and Hongning Wang. 2021. Category-aware collaborative sequential recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 388–397.
- [9] Chunlu Chen, Ji Liu, Haowen Tan, Xingjian Li, Kevin I-Kai Wang, Peng Li, Kouichi Sakurai, and Dejing Dou. 2025. Trustworthy federated learning: privacy, security, and beyond. *Knowledge and Information Systems* 67, 3 (2025), 2321–2356.
- [10] Wanyu Chen, Fei Cai, Honghui Chen, and Maarten De Rijke. 2019. Joint neural collaborative filtering for recommender systems. *ACM Transactions on Information Systems (TOIS)* 37, 4 (2019), 1–30.
- [11] Zefeng Chen, Wensheng Gan, Jiayang Wu, Kaixia Hu, and Hong Lin. 2025. Data scarcity in recommendation systems: A survey. *ACM Transactions on Recommender Systems* 3, 3 (2025), 1–31.
- [12] Sang-Min Choi and Yo-Sub Han. 2010. A content recommendation system based on category correlations. In *2010 Fifth International Multi-conference on Computing in the Global Information Technology*. IEEE, 66–70.
- [13] Kenneth Lee Clarkson. 1985. *Algorithms for Closest-Point Problems (Computational Geometry)*. Ph. D. Dissertation. Stanford University, Stanford, CA, USA. Advisor(s) Yao, Andrew C. AAT 8506171.
- [14] Yashar Deldjoo, Tommaso Di Noia, and Felice Antonio Merri. 2021. A survey on adversarial recommender systems: from attack/defense strategies to generative adversarial networks. *ACM Computing Surveys (CSUR)* 54, 2 (2021), 1–38.
- [15] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Gong. 2020. Local model poisoning attacks to {Byzantine-Robust} federated learning. In *29th USENIX security symposium (USENIX Security 20)*. 1605–1622.
- [16] Minghong Fang, Neil Zhenqiang Gong, and Jia Liu. 2020. Influence function based data poisoning attacks to top-n recommender systems. In *Proceedings of The Web Conference 2020*. 3019–3025.
- [17] Rachid Guerraoui, Sébastien Rouault, et al. 2018. The hidden vulnerability of distributed learning in byzantium. In *International Conference on Machine Learning*. PMLR, 3521–3530.
- [18] Asela Gunawardana and Guy Shani. 2009. A survey of accuracy evaluation metrics of recommendation tasks. *Journal of Machine Learning Research* 10, 12 (2009).
- [19] Yurong Hao, Xihui Chen, Wei Wang, Jiqiang Liu, Tao Li, Junyong Wang, and Witold Pedrycz. 2024. Eyes on Federated Recommendation: Targeted Poisoning With Competition and Its Mitigation. *IEEE Transactions on Information Forensics and Security* (2024).
- [20] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web (Perth, Australia) (WWW '17)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 173–182. doi:10.1145/3038912.3052569
- [21] Hai Huang, Jiaming Mu, Neil Zhenqiang Gong, Qi Li, Bin Liu, and Mingwei Xu. 2021. Data poisoning attacks to deep learning based recommender systems. *arXiv preprint arXiv:2101.02644* (2021).
- [22] Yan Jia, Zhaoquan Gu, Lei Du, Yu Long, Ye Wang, Jianxin Li, and Yanchun Zhang. 2023. Artificial intelligence enabled cyber security defense for smart cities: A novel attack detection framework based on the MDATA model. *Knowledge-Based Systems* 276 (2023), 110781.
- [23] Xueyong Jiang, Baisong Liu, Jiangchen Qin, Yunchong Zhang, and Jiangbo Qian. 2022. FedNCF: federated neural collaborative filtering for privacy-preserving recommender system. In *2022 International joint conference on neural networks (IJCNN)*. IEEE, 1–8.
- [24] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

- [25] Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [26] Zitao Li, Bolin Ding, Ce Zhang, Ninghui Li, and Jingren Zhou. 2021. Federated matrix factorization with privacy guarantee. *Proceedings of the VLDB Endowment* 15, 4 (2021).
- [27] Guanyu Lin, Feng Liang, Weike Pan, and Zhong Ming. 2020. Fedrec: Federated recommendation with explicit feedback. *IEEE Intelligent Systems* 36, 5 (2020), 21–30.
- [28] Yujie Lin, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Dongxiao Yu, Jun Ma, Maarten de Rijke, and Xiuzhen Cheng. 2020. Meta matrix factorization for federated rating predictions. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 981–990.
- [29] Fengxia Liu, Zhiming Zheng, Yexuan Shi, Yongxin Tong, and Yi Zhang. 2024. A survey on federated learning: a perspective from multi-party computation. *Frontiers of Computer Science* 18, 1 (2024), 181336.
- [30] Shuchang Liu, Yingqiang Ge, Shuyuan Xu, Yongfeng Zhang, and Amelie Marian. 2022. Fairness-aware federated matrix factorization. In *Proceedings of the 16th ACM conference on recommender systems*. 168–178.
- [31] Zhiwei Liu, Liangwei Yang, Ziwei Fan, Hao Peng, and Philip S Yu. 2022. Federated social recommendation with graph neural network. *ACM Transactions on Intelligent Systems and Technology (TIST)* 13, 4 (2022), 1–24.
- [32] Hongqin Lyu, Yongxiong Zhang, Chao Wang, Shigong Long, and Shengnan Guo. 2023. Federated learning privacy incentives: Reverse auctions and negotiations. *CAAI Transactions on Intelligence Technology* 8, 4 (2023), 1538–1557.
- [33] Khalil Muhammad, Qinqin Wang, Diarmuid O'Reilly-Morgan, Elias Tragos, Barry Smyth, Neil Hurley, James Geraci, and Aonghus Lawlor. 2020. Fedfast: Going beyond average for faster training of federated recommender systems. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1234–1242.
- [34] Thanh Toan Nguyen, Nguyen Quoc Viet Hung, Thanh Tam Nguyen, Thanh Trung Huynh, Thanh Thi Nguyen, Matthias Weidlich, and Hongzhi Yin. 2024. Manipulating recommender systems: A survey of poisoning attacks and countermeasures. *Comput. Surveys* (2024).
- [35] Toan Nguyen Thanh, Nguyen Duc Khang Quach, Thanh Tam Nguyen, Thanh Trung Huynh, Viet Hung Vu, Phi Le Nguyen, Jun Jo, and Quoc Viet Hung Nguyen. 2023. Poisoning GNN-based Recommender Systems with Generative Surrogate-based Attacks. *ACM Trans. Inf. Syst.* 41, 3, Article 58 (Feb. 2023), 24 pages. doi:10.1145/3567420
- [36] Vasileios Perifanis and Pavlos S Efrimidis. 2022. Federated neural collaborative filtering. *Knowledge-Based Systems* 242 (2022), 108441.
- [37] Dazhong Rong, Qinming He, and Jianhai Chen. 2022. Poisoning deep learning based recommender model in federated learning scenarios. *arXiv preprint arXiv:2204.13594* (2022).
- [38] Dazhong Rong, Shuai Ye, Ruoyan Zhao, Hon Ning Yuen, Jianhai Chen, and Qinming He. 2022. Fedrecattack: Model poisoning attack to federated recommendation. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2643–2655.
- [39] Alexander J Smola, A Gretton, and K Borgwardt. 2006. Maximum mean discrepancy. In *13th international conference, ICONIP*, Vol. 6.
- [40] Harald Steck. 2011. Item popularity and recommendation accuracy. In *Proceedings of the fifth ACM conference on Recommender systems*. 125–132.
- [41] Jiajie Su, Chaochao Chen, Weiming Liu, Zibin Lin, Shuheng Shen, Weiqiang Wang, and Xiaolin Zheng. 2024. Revisit targeted model poisoning on federated recommendation: Optimize via multi-objective transport. In *Proceedings of the 47th international acm sigir conference on research and development in information retrieval*. 1722–1732.
- [42] Jiajie Su, Chaochao Chen, Yihao Wang, Weiming Liu, Yuyuan Li, Tao Wang, Zhigang Li, Xiaolin Zheng, and Jianwei Yin. 2025. DuAda: Adaptive Targeted Model Poisoning Attack Framework via Dummy User Simulation on Federated Recommendation. *ACM Trans. Inf. Syst.* 43, 6, Article 161 (Sept. 2025), 37 pages. doi:10.1145/3757059
- [43] Chuan Sun, Xiuhua Li, Junhao Wen, Xiaofei Wang, Zhu Han, and Victor CM Leung. 2023. Federated deep reinforcement learning for recommendation-enabled edge caching in mobile edge-cloud computing networks. *IEEE Journal on Selected Areas in Communications* 41, 3 (2023), 690–705.
- [44] Ziteng Sun, Peter Kairouz, Ananda Theertha Suresh, and H Brendan McMahan. 2019. Can you really backdoor federated learning? *arXiv preprint arXiv:1911.07963* (2019).
- [45] Hind Taud and Jean-Francois Mas. 2017. Multilayer perceptron (MLP). In *Geomatic approaches for modeling land change scenarios*. Springer, 451–455.
- [46] Vale Tolpegin, Stacey Truex, Mehmet Emre Gursay, and Ling Liu. 2020. Data poisoning attacks against federated learning systems. In *Computer security—ESORICS 2020: 25th European symposium on research in computer security, ESORICS 2020, guildford, UK, September 14–18, 2020, proceedings, part i* 25. Springer, 480–501.
- [47] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A practical guide*, 1st ed., Cham: Springer International Publishing 10, 3152676 (2017), 10–5555.
- [48] Omar Abdel Wahab, Gaith Rjoub, Jamal Bentahar, and Robin Cohen. 2022. Federated against the cold: A trust-based federated learning approach to counter the cold start problem in recommendation systems. *Information Sciences* 601 (2022), 189–206.

- [49] Chenwang Wu, Defu Lian, Yong Ge, Zhihao Zhu, and Enhong Chen. 2023. Influence-driven data poisoning for robust recommender systems. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 10 (2023), 11915–11931.
- [50] Chuhan Wu, Fangzhao Wu, Yang Cao, Yongfeng Huang, and Xing Xie. 2021. Fedgnn: Federated graph neural network for privacy-preserving recommendation. *arXiv preprint arXiv:2102.04925* (2021).
- [51] Chuhan Wu, Fangzhao Wu, Tao Qi, Yongfeng Huang, and Xing Xie. 2022. FedAttack: Effective and covert poisoning attack on federated recommendation via hard sampling. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4164–4172.
- [52] Chuhan Wu, Fangzhao Wu, Tao Qi, Yongfeng Huang, and Xing Xie. 2022. Fedcl: Federated contrastive learning for privacy-preserving recommendation. *arXiv preprint arXiv:2204.09850* (2022).
- [53] Hong-Jian Xue, Xinyu Dai, Jianbing Zhang, Shujian Huang, and Jiajun Chen. 2017. Deep matrix factorization models for recommender systems.. In *IJCAI*, Vol. 17. Melbourne, Australia, 3203–3209.
- [54] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. 2018. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International conference on machine learning*. Pmlr, 5650–5659.
- [55] Ming Yin, Yichang Xu, Minghong Fang, and Neil Zhenqiang Gong. 2024. Poisoning Federated Recommender Systems with Fake Users. In *Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) (WWW '24). New York, NY, USA, 3555–3565. doi:10.1145/3589334.3645492
- [56] Yang Yu, Qi Liu, Likang Wu, Runlong Yu, Sanshi Lei Yu, and Zaixi Zhang. 2023. Untargeted attack against federated recommendation systems via poisonous item embeddings and the defense. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 4854–4863.
- [57] Wei Yuan, Quoc Viet Hung Nguyen, Tiek He, Liang Chen, and Hongzhi Yin. 2023. Manipulating Federated Recommender Systems: Poisoning with Synthetic Users and Its Countermeasures. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Taipei, Taiwan) (SIGIR '23). New York, NY, USA, 1690–1699. doi:10.1145/3539618.3591722
- [58] Wei Yuan, Chaoqun Yang, Liang Qu, Quoc Viet Hung Nguyen, Guanhua Ye, and Hongzhi Yin. 2025. PTF-FSR: A Parameter Transmission-Free Federated Sequential Recommender System. *ACM Trans. Inf. Syst.* 43, 2, Article 52 (Jan. 2025), 24 pages. doi:10.1145/3708344
- [59] Wei Yuan, Chaoqun Yang, Liang Qu, Guanhua Ye, Quoc Viet Hung Nguyen, and Hongzhi Yin. 2025. Robust federated contrastive recommender system against targeted model poisoning attack. *Science China Information Sciences* 68, 4 (2025), 1–16.
- [60] Wei Yuan, Shilong Yuan, Chaoqun Yang, Nguyen Quoc Viet hung, and Hongzhi Yin. 2023. Manipulating visually aware federated recommender systems and its countermeasures. *ACM Transactions on Information Systems* 42, 3 (2023), 1–26.
- [61] Hengtong Zhang, Changxin Tian, Yaliang Li, Lu Su, Nan Yang, Wayne Xin Zhao, and Jing Gao. 2021. Data poisoning attack against recommender system using incomplete and perturbed data. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2154–2164.
- [62] Jun Zhang, Huan Li, Dazhong Rong, Yan Zhao, Ke Chen, and Lidan Shou. 2024. Preventing the Popular Item Embedding Based Attack in Federated Recommendations. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2179–2191.
- [63] Shijie Zhang, Hongzhi Yin, Tong Chen, Zi Huang, Quoc Viet Hung Nguyen, and Lizhen Cui. 2022. Pipattack: Poisoning federated recommender systems for manipulating item promotion. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1415–1423.
- [64] Xiao Zhang, Ziming Ye, Jianfeng Lu, Fuzhen Zhuang, Yanwei Zheng, and Dongxiao Yu. 2023. Fine-grained preference-aware personalized federated poi recommendation with data sparsity. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 413–422.
- [65] Yuying Zhao, Yu Wang, Yunchao Liu, Xueqi Cheng, Charu C Aggarwal, and Tyler Derr. 2025. Fairness and diversity in recommender systems: a survey. *ACM Transactions on Intelligent Systems and Technology* 16, 1 (2025), 1–28.